Predicting Company Bankruptcies

Introduction

In today's highly competitive and dynamic economic environment, companies face increasing financial pressure due to market volatility, rising operational costs, and uncertain economic conditions. As a result, corporate bankruptcy has become a significant concern for financial institutions, investors, and regulatory bodies. Early identification of financially distressed companies is essential to reduce financial risks and support informed decision-making.

This project focuses on developing a machine learning–based system to predict company bankruptcies using historical financial data and key financial ratios. By analyzing patterns related to profitability, liquidity, leverage, and cash flow, the proposed model aims to classify companies as bankrupt or non-bankrupt with high accuracy. The solution assists organizations in improving risk assessment, optimizing investment strategies, and enhancing overall financial stability through data-driven insights.

Problem Statement

Corporate bankruptcy poses a major risk to financial institutions, investors, and stakeholders, as unexpected business failures can lead to significant financial losses and economic instability. Traditional methods of assessing bankruptcy risk often rely on manual analysis and fixed financial rules, which may fail to capture complex relationships among financial indicators and are unable to adapt to changing market conditions.

The objective of this project is to design and implement a robust machine learning–based classification system that accurately predicts whether a company is likely to go bankrupt based on its financial ratios. The model must handle class imbalance, select the most relevant financial features, and evaluate multiple machine learning algorithms to identify the most effective approach. By providing reliable bankruptcy predictions, the system aims to support data-driven risk assessment and informed financial decision-making.

Problem Type

This is a binary classification problem where the objective is to predict whether a company will go bankrupt (1) or not bankrupt (0) based on financial ratios.

Project Workflow

- Step 1 | Import Required Libraries
- Step 2 | Data Loading
- Step 3 | Dataset Overview & Understanding
- Step 4 | Target Variable Analysis
- Step 5 | Exploratory Data Analysis (EDA)
- Step 6 | Correlation Analysis & Insights
- Step 7 | Feature Selection
- Step 8 | Data Preprocessing
- Step 9 | Feature Scaling & Train-Test Split
- Step 10 | Logistic Regression Model
- Step 11 | Decision Tree Model
- Step 12 | Random Forest Model
- Step 13 | Gradient Boosting Model
- Step 14 | XGBoost Model (Optional)
- Step 15 | Model Evaluation, Comparison & Conclusion

Let's get started:

Import Libraries

Step 1 | Import Required Libraries

In this step, essential Python libraries are imported to perform data manipulation, visualization, and machine learning tasks. Libraries such as NumPy and Pandas are used for handling and analyzing transaction data, while Matplotlib and Seaborn help visualize patterns and trends. Scikit-learn provides tools for preprocessing, model building, and evaluation.

In [2]:
```
import numpy as np
import pandas as pd
import os

import matplotlib.pyplot as plt
import seaborn as sns
from matplotlib.colors import ListedColormap

from sklearn.model_selection import train_test_split
from scipy.stats import boxcox
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import StandardScaler,OneHotEncoder
```

```
from sklearn.model_selection import GridSearchCV, StratifiedKFold

from sklearn.svm import SVC
from sklearn.tree import DecisionTreeClassifier
from sklearn.ensemble import RandomForestClassifier
from sklearn.linear_model import LogisticRegression
from sklearn.compose import ColumnTransformer
from sklearn.preprocessing import LabelEncoder

from sklearn.metrics import classification_report, accuracy_score, confusion_matrix

import ipywidgets as widgets
from IPython.display import display, HTML
import joblib
```

In [3]:
```
os.getcwd()
```

Out[3]:
```
'C:\\Users\\ajoyb'
```

In [4]:
```
os.chdir(r"C:\BLA Capston Projects")
```

In [5]:
```
os.getcwd()
```

Out[5]:
```
'C:\\BLA Capston Projects'
```

Read Dataset

In [6]:
```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.preprocessing import StandardScaler
from sklearn.cluster import KMeans
from sklearn.model_selection import train_test_split
```

```
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import accuracy_score, classification_report
```

In [7]:
```
df=pd.read_csv("Predicting Company Bankruptcies.csv")
```

In [8]:
```
df
```

Out[8]:

	Bankrupt?	ROA(C) before interest and depreciation before interest	ROA(A) before interest and % after tax	ROA(B) before interest and depreciation after tax	Operating Gross Margin	Realized Sales Gross Margin	Operating Profit Rate	Pre-tax net Interest Rate	After-tax net Interest Rate	Non-industry income and expenditure/revenue	...	Inc to T As
0	1	0.370594	0.424389	0.405750	0.601457	0.601457	0.998969	0.796887	0.808809	0.302646	...	0.716
1	1	0.464291	0.538214	0.516730	0.610235	0.610235	0.998946	0.797380	0.809301	0.303556	...	0.795
2	1	0.426071	0.499019	0.472295	0.601450	0.601364	0.998857	0.796403	0.808388	0.302035	...	0.774
3	1	0.399844	0.451265	0.457733	0.583541	0.583541	0.998700	0.796967	0.808966	0.303350	...	0.739
4	1	0.465022	0.538432	0.522298	0.598783	0.598783	0.998973	0.797366	0.809304	0.303475	...	0.795
...	...	...	...	...	...	...	...	...	...	...	...	
6814	0	0.493687	0.539468	0.543230	0.604455	0.604462	0.998992	0.797409	0.809331	0.303510	...	0.799
6815	0	0.475162	0.538269	0.524172	0.598308	0.598308	0.998992	0.797414	0.809327	0.303520	...	0.799
6816	0	0.472725	0.533744	0.520638	0.610444	0.610213	0.998984	0.797401	0.809317	0.303512	...	0.797
6817	0	0.506264	0.559911	0.554045	0.607850	0.607850	0.999074	0.797500	0.809399	0.303498	...	0.811
6818	0	0.493053	0.570105	0.549548	0.627409	0.627409	0.998080	0.801987	0.813800	0.313415	...	0.815

6819 rows × 96 columns

Dataset Description

Data Summary & Description

The dataset used in this project contains financial information of companies and is designed to predict corporate bankruptcy. Each record represents a company described by multiple financial ratios related to profitability, liquidity, leverage, and operational efficiency. The dataset is used to analyze financial health and classify companies as bankrupt or non-bankrupt using machine learning techniques.

Descriptive Statistics for Numerical Features

Profitability Ratios	Measure a company's ability to generate profit relative to revenue, assets, or equity
Liquidity Ratios	Indicate a company's ability to meet short-term financial obligations
Leverage Ratios	Reflect the extent of debt used in financing company assets
Cash Flow Indicators	Show the efficiency of cash generation from business operations
Operational Efficiency	Evaluate how effectively a company utilizes its resources

Variable Descriptions (Bankruptcy Dataset)

Financial Ratios	Multiple numerical features representing liquidity, profitability, leverage, and cash flow metrics of companies
Company Records	Each row corresponds to the financial profile of a single company
Total Features	95 financial indicators used as input variables
Bankrupt?	Target Variable indicating company bankruptcy status 1 = Bankrupt Company 0 = Non-Bankrupt Company

Step 2 | Data Loading

Data Loading

Loading the Dataset

In this step, the bankruptcy prediction dataset is loaded into the Python environment using the Pandas library. The dataset is stored in a CSV file format and contains financial information of multiple companies. Proper data loading is essential to ensure that all features and target variables are correctly imported for further analysis.

Description

- **pd.read_csv()** is used to read the dataset from a CSV file into a Pandas DataFrame.
- The **encoding="latin1"** parameter ensures that special characters are handled correctly during data loading.
- The dataset is stored in the variable **df**, which allows easy access for data exploration and preprocessing.
- **df.head()** displays the first five records of the dataset to verify successful loading and understand the data structure.

After loading the dataset, it is ready for further steps such as exploratory data analysis, preprocessing, feature selection, and model training.

Now A Deep Dive into Transaction Patterns:

In [9]:
```
df
```

Out[9]:

	Bankrupt?	ROA(C) before interest and depreciation before interest	ROA(A) before interest and % after tax	ROA(B) before interest and depreciation after tax	Operating Gross Margin	Realized Sales Gross Margin	Operating Profit Rate	Pre-tax net Interest Rate	After-tax net Interest Rate	Non-industry income and expenditure/revenue	...	Inc to T As
0	1	0.370594	0.424389	0.405750	0.601457	0.601457	0.998969	0.796887	0.808809	0.302646	...	0.716
1	1	0.464291	0.538214	0.516730	0.610235	0.610235	0.998946	0.797380	0.809301	0.303556	...	0.795
2	1	0.426071	0.499019	0.472295	0.601450	0.601364	0.998857	0.796403	0.808388	0.302035	...	0.774
3	1	0.399844	0.451265	0.457733	0.583541	0.583541	0.998700	0.796967	0.808966	0.303350	...	0.739
4	1	0.465022	0.538432	0.522298	0.598783	0.598783	0.998973	0.797366	0.809304	0.303475	...	0.795
...	...	...	...	...	...	...	...	...	...	...	...	
6814	0	0.493687	0.539468	0.543230	0.604455	0.604462	0.998992	0.797409	0.809331	0.303510	...	0.799
6815	0	0.475162	0.538269	0.524172	0.598308	0.598308	0.998992	0.797414	0.809327	0.303520	...	0.799
6816	0	0.472725	0.533744	0.520638	0.610444	0.610213	0.998984	0.797401	0.809317	0.303512	...	0.797
6817	0	0.506264	0.559911	0.554045	0.607850	0.607850	0.999074	0.797500	0.809399	0.303498	...	0.811
6818	0	0.493053	0.570105	0.549548	0.627409	0.627409	0.998080	0.801987	0.813800	0.313415	...	0.815

6819 rows × 96 columns

In [10]:
```
df.head()
```

Out[10]:

	Bankrupt?	ROA(C) before interest and depreciation before interest	ROA(A) before interest and % after tax	ROA(B) before interest and depreciation after tax	Operating Gross Margin	Realized Sales Gross Margin	Operating Profit Rate	Pre-tax net Interest Rate	After-tax net Interest Rate	Non-industry income and expenditure/revenue	...	Net Income to Total Assets
0	1	0.370594	0.424389	0.405750	0.601457	0.601457	0.998969	0.796887	0.808809	0.302646	...	0.716845
1	1	0.464291	0.538214	0.516730	0.610235	0.610235	0.998946	0.797380	0.809301	0.303556	...	0.795297
2	1	0.426071	0.499019	0.472295	0.601450	0.601364	0.998857	0.796403	0.808388	0.302035	...	0.774670
3	1	0.399844	0.451265	0.457733	0.583541	0.583541	0.998700	0.796967	0.808966	0.303350	...	0.739555
4	1	0.465022	0.538432	0.522298	0.598783	0.598783	0.998973	0.797366	0.809304	0.303475	...	0.795016

5 rows × 96 columns

In [11]:
```
df.shape
```

Out[11]:
```
(6819, 96)
```

In [12]:
```
df.columns
```

Out[12]:
```
Index(['Bankrupt?', ' ROA(C) before interest and depreciation before interest',
       ' ROA(A) before interest and % after tax',
       ' ROA(B) before interest and depreciation after tax',
       ' Operating Gross Margin', ' Realized Sales Gross Margin',
       ' Operating Profit Rate', ' Pre-tax net Interest Rate',
       ' After-tax net Interest Rate',
       ' Non-industry income and expenditure/revenue',
       ' Continuous interest rate (after tax)', ' Operating Expense Rate',
       ' Research and development expense rate', ' Cash flow rate',
       ' Interest-bearing debt interest rate', ' Tax rate (A)',
       ' Net Value Per Share (B)', ' Net Value Per Share (A)',
       ' Net Value Per Share (C)', ' Persistent EPS in the Last Four Seasons',
       ' Cash Flow Per Share', ' Revenue Per Share (Yuan ¥)',
       ' Operating Profit Per Share (Yuan ¥)',
       ' Per Share Net profit before tax (Yuan ¥)',
       ' Realized Sales Gross Profit Growth Rate',
       ' Operating Profit Growth Rate', ' After-tax Net Profit Growth Rate',
       ' Regular Net Profit Growth Rate', ' Continuous Net Profit Growth Rate',
       ' Total Asset Growth Rate', ' Net Value Growth Rate',
       ' Total Asset Return Growth Rate Ratio', ' Cash Reinvestment %',
       ' Current Ratio', ' Quick Ratio', ' Interest Expense Ratio',
       ' Total debt/Total net worth', ' Debt ratio %', ' Net worth/Assets',
       ' Long-term fund suitability ratio (A)', ' Borrowing dependency',
       ' Contingent liabilities/Net worth',
       ' Operating profit/Paid-in capital',
       ' Net profit before tax/Paid-in capital',
       ' Inventory and accounts receivable/Net value', ' Total Asset Turnover',
       ' Accounts Receivable Turnover', ' Average Collection Days',
       ' Inventory Turnover Rate (times)', ' Fixed Assets Turnover Frequency',
       ' Net Worth Turnover Rate (times)', ' Revenue per person',
       ' Operating profit per person', ' Allocation rate per person',
       ' Working Capital to Total Assets', ' Quick Assets/Total Assets',
       ' Current Assets/Total Assets', ' Cash/Total Assets',
       ' Quick Assets/Current Liability', ' Cash/Current Liability',
       ' Current Liability to Assets', ' Operating Funds to Liability',
       ' Inventory/Working Capital', ' Inventory/Current Liability',
       ' Current Liabilities/Liability', ' Working Capital/Equity',
       ' Current Liabilities/Equity', ' Long-term Liability to Current Assets',
       ' Retained Earnings to Total Assets', ' Total income/Total expense',
       ' Total expense/Assets', ' Current Asset Turnover Rate',
```

```
       ' Quick Asset Turnover Rate', ' Working capitcal Turnover Rate',
       ' Cash Turnover Rate', ' Cash Flow to Sales', ' Fixed Assets to Assets',
       ' Current Liability to Liability', ' Current Liability to Equity',
       ' Equity to Long-term Liability', ' Cash Flow to Total Assets',
       ' Cash Flow to Liability', ' CFO to Assets', ' Cash Flow to Equity',
       ' Current Liability to Current Assets', ' Liability-Assets Flag',
       ' Net Income to Total Assets', ' Total assets to GNP price',
       ' No-credit Interval', ' Gross Profit to Sales',
       ' Net Income to Stockholder's Equity', ' Liability to Equity',
       ' Degree of Financial Leverage (DFL)',
       ' Interest Coverage Ratio (Interest expense to EBIT)',
       ' Net Income Flag', ' Equity to Liability'],
      dtype='object')
```

In [13]:
```
df['Bankrupt?'].value_counts()
```

Out[13]:
```
Bankrupt?
0    6599
1     220
Name: count, dtype: int64
```

Step 3 | Dataset Overview & Understanding

In [14]:
```
df.shape
```

Out[14]:
```
(6819, 96)
```

In [15]:
```
df.columns
```

Out[15]:
```
Index(['Bankrupt?', ' ROA(C) before interest and depreciation before interest',
       ' ROA(A) before interest and % after tax',
       ' ROA(B) before interest and depreciation after tax',
       ' Operating Gross Margin', ' Realized Sales Gross Margin',
       ' Operating Profit Rate', ' Pre-tax net Interest Rate',
       ' After-tax net Interest Rate',
       ' Non-industry income and expenditure/revenue',
       ' Continuous interest rate (after tax)', ' Operating Expense Rate',
       ' Research and development expense rate', ' Cash flow rate',
       ' Interest-bearing debt interest rate', ' Tax rate (A)',
       ' Net Value Per Share (B)', ' Net Value Per Share (A)',
       ' Net Value Per Share (C)', ' Persistent EPS in the Last Four Seasons',
       ' Cash Flow Per Share', ' Revenue Per Share (Yuan ¥)',
       ' Operating Profit Per Share (Yuan ¥)',
       ' Per Share Net profit before tax (Yuan ¥)',
       ' Realized Sales Gross Profit Growth Rate',
       ' Operating Profit Growth Rate', ' After-tax Net Profit Growth Rate',
       ' Regular Net Profit Growth Rate', ' Continuous Net Profit Growth Rate',
       ' Total Asset Growth Rate', ' Net Value Growth Rate',
       ' Total Asset Return Growth Rate Ratio', ' Cash Reinvestment %',
       ' Current Ratio', ' Quick Ratio', ' Interest Expense Ratio',
       ' Total debt/Total net worth', ' Debt ratio %', ' Net worth/Assets',
       ' Long-term fund suitability ratio (A)', ' Borrowing dependency',
       ' Contingent liabilities/Net worth',
       ' Operating profit/Paid-in capital',
       ' Net profit before tax/Paid-in capital',
       ' Inventory and accounts receivable/Net value', ' Total Asset Turnover',
       ' Accounts Receivable Turnover', ' Average Collection Days',
       ' Inventory Turnover Rate (times)', ' Fixed Assets Turnover Frequency',
       ' Net Worth Turnover Rate (times)', ' Revenue per person',
       ' Operating profit per person', ' Allocation rate per person',
       ' Working Capital to Total Assets', ' Quick Assets/Total Assets',
       ' Current Assets/Total Assets', ' Cash/Total Assets',
       ' Quick Assets/Current Liability', ' Cash/Current Liability',
       ' Current Liability to Assets', ' Operating Funds to Liability',
       ' Inventory/Working Capital', ' Inventory/Current Liability',
       ' Current Liabilities/Liability', ' Working Capital/Equity',
       ' Current Liabilities/Equity', ' Long-term Liability to Current Assets',
       ' Retained Earnings to Total Assets', ' Total income/Total expense',
       ' Total expense/Assets', ' Current Asset Turnover Rate',
```

```
       ' Quick Asset Turnover Rate', ' Working capitcal Turnover Rate',
       ' Cash Turnover Rate', ' Cash Flow to Sales', ' Fixed Assets to Assets',
       ' Current Liability to Liability', ' Current Liability to Equity',
       ' Equity to Long-term Liability', ' Cash Flow to Total Assets',
       ' Cash Flow to Liability', ' CFO to Assets', ' Cash Flow to Equity',
       ' Current Liability to Current Assets', ' Liability-Assets Flag',
       ' Net Income to Total Assets', ' Total assets to GNP price',
       ' No-credit Interval', ' Gross Profit to Sales',
       ' Net Income to Stockholder's Equity', ' Liability to Equity',
       ' Degree of Financial Leverage (DFL)',
       ' Interest Coverage Ratio (Interest expense to EBIT)',
       ' Net Income Flag', ' Equity to Liability'],
      dtype='object')
```

In [16]:
```
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 6819 entries, 0 to 6818
Data columns (total 96 columns):
 #   Column                                                    Non-Null Count  Dtype
---  ------                                                    --------------  -----
 0   Bankrupt?                                                 6819 non-null   int64
 1    ROA(C) before interest and depreciation before interest  6819 non-null   float64
 2    ROA(A) before interest and % after tax                   6819 non-null   float64
 3    ROA(B) before interest and depreciation after tax        6819 non-null   float64
 4    Operating Gross Margin                                   6819 non-null   float64
 5    Realized Sales Gross Margin                              6819 non-null   float64
 6    Operating Profit Rate                                    6819 non-null   float64
 7    Pre-tax net Interest Rate                                6819 non-null   float64
 8    After-tax net Interest Rate                              6819 non-null   float64
 9    Non-industry income and expenditure/revenue              6819 non-null   float64
 10   Continuous interest rate (after tax)                     6819 non-null   float64
 11   Operating Expense Rate                                   6819 non-null   float64
 12   Research and development expense rate                    6819 non-null   float64
 13   Cash flow rate                                           6819 non-null   float64
 14   Interest-bearing debt interest rate                      6819 non-null   float64
 15   Tax rate (A)                                             6819 non-null   float64
 16   Net Value Per Share (B)                                  6819 non-null   float64
 17   Net Value Per Share (A)                                  6819 non-null   float64
 18   Net Value Per Share (C)                                  6819 non-null   float64
 19   Persistent EPS in the Last Four Seasons                  6819 non-null   float64
 20   Cash Flow Per Share                                      6819 non-null   float64
 21   Revenue Per Share (Yuan ¥)                               6819 non-null   float64
 22   Operating Profit Per Share (Yuan ¥)                      6819 non-null   float64
 23   Per Share Net profit before tax (Yuan ¥)                 6819 non-null   float64
 24   Realized Sales Gross Profit Growth Rate                  6819 non-null   float64
 25   Operating Profit Growth Rate                             6819 non-null   float64
 26   After-tax Net Profit Growth Rate                         6819 non-null   float64
 27   Regular Net Profit Growth Rate                           6819 non-null   float64
 28   Continuous Net Profit Growth Rate                        6819 non-null   float64
 29   Total Asset Growth Rate                                  6819 non-null   float64
 30   Net Value Growth Rate                                    6819 non-null   float64
 31   Total Asset Return Growth Rate Ratio                     6819 non-null   float64
 32   Cash Reinvestment %                                      6819 non-null   float64
 33   Current Ratio                                            6819 non-null   float64
 34   Quick Ratio                                              6819 non-null   float64
 35   Interest Expense Ratio                                   6819 non-null   float64
```

36	Total debt/Total net worth	6819	non-null	float64
37	Debt ratio %	6819	non-null	float64
38	Net worth/Assets	6819	non-null	float64
39	Long-term fund suitability ratio (A)	6819	non-null	float64
40	Borrowing dependency	6819	non-null	float64
41	Contingent liabilities/Net worth	6819	non-null	float64
42	Operating profit/Paid-in capital	6819	non-null	float64
43	Net profit before tax/Paid-in capital	6819	non-null	float64
44	Inventory and accounts receivable/Net value	6819	non-null	float64
45	Total Asset Turnover	6819	non-null	float64
46	Accounts Receivable Turnover	6819	non-null	float64
47	Average Collection Days	6819	non-null	float64
48	Inventory Turnover Rate (times)	6819	non-null	float64
49	Fixed Assets Turnover Frequency	6819	non-null	float64
50	Net Worth Turnover Rate (times)	6819	non-null	float64
51	Revenue per person	6819	non-null	float64
52	Operating profit per person	6819	non-null	float64
53	Allocation rate per person	6819	non-null	float64
54	Working Capital to Total Assets	6819	non-null	float64
55	Quick Assets/Total Assets	6819	non-null	float64
56	Current Assets/Total Assets	6819	non-null	float64
57	Cash/Total Assets	6819	non-null	float64
58	Quick Assets/Current Liability	6819	non-null	float64
59	Cash/Current Liability	6819	non-null	float64
60	Current Liability to Assets	6819	non-null	float64
61	Operating Funds to Liability	6819	non-null	float64
62	Inventory/Working Capital	6819	non-null	float64
63	Inventory/Current Liability	6819	non-null	float64
64	Current Liabilities/Liability	6819	non-null	float64
65	Working Capital/Equity	6819	non-null	float64
66	Current Liabilities/Equity	6819	non-null	float64
67	Long-term Liability to Current Assets	6819	non-null	float64
68	Retained Earnings to Total Assets	6819	non-null	float64
69	Total income/Total expense	6819	non-null	float64
70	Total expense/Assets	6819	non-null	float64
71	Current Asset Turnover Rate	6819	non-null	float64
72	Quick Asset Turnover Rate	6819	non-null	float64
73	Working capitcal Turnover Rate	6819	non-null	float64
74	Cash Turnover Rate	6819	non-null	float64
75	Cash Flow to Sales	6819	non-null	float64
76	Fixed Assets to Assets	6819	non-null	float64

```
 77   Current Liability to Liability                          6819 non-null   float64
 78   Current Liability to Equity                             6819 non-null   float64
 79   Equity to Long-term Liability                           6819 non-null   float64
 80   Cash Flow to Total Assets                               6819 non-null   float64
 81   Cash Flow to Liability                                  6819 non-null   float64
 82   CFO to Assets                                           6819 non-null   float64
 83   Cash Flow to Equity                                     6819 non-null   float64
 84   Current Liability to Current Assets                     6819 non-null   float64
 85   Liability-Assets Flag                                   6819 non-null   int64
 86   Net Income to Total Assets                              6819 non-null   float64
 87   Total assets to GNP price                               6819 non-null   float64
 88   No-credit Interval                                      6819 non-null   float64
 89   Gross Profit to Sales                                   6819 non-null   float64
 90   Net Income to Stockholder's Equity                      6819 non-null   float64
 91   Liability to Equity                                     6819 non-null   float64
 92   Degree of Financial Leverage (DFL)                      6819 non-null   float64
 93   Interest Coverage Ratio (Interest expense to EBIT)      6819 non-null   float64
 94   Net Income Flag                                         6819 non-null   int64
 95   Equity to Liability                                     6819 non-null   float64
dtypes: float64(93), int64(3)
memory usage: 5.0 MB
```

In [17]:
```
df.head()
```

Out[17]:

	Bankrupt?	ROA(C) before interest and depreciation before interest	ROA(A) before interest and % after tax	ROA(B) before interest and depreciation after tax	Operating Gross Margin	Realized Sales Gross Margin	Operating Profit Rate	Pre-tax net Interest Rate	After-tax net Interest Rate	Non-industry income and expenditure/revenue	...	Net Income to Total Assets
0	1	0.370594	0.424389	0.405750	0.601457	0.601457	0.998969	0.796887	0.808809	0.302646	...	0.716845
1	1	0.464291	0.538214	0.516730	0.610235	0.610235	0.998946	0.797380	0.809301	0.303556	...	0.795297
2	1	0.426071	0.499019	0.472295	0.601450	0.601364	0.998857	0.796403	0.808388	0.302035	...	0.774670
3	1	0.399844	0.451265	0.457733	0.583541	0.583541	0.998700	0.796967	0.808966	0.303350	...	0.739555
4	1	0.465022	0.538432	0.522298	0.598783	0.598783	0.998973	0.797366	0.809304	0.303475	...	0.795016

5 rows × 96 columns

In [18]:
```
df.tail()
```

Out[18]:

	Bankrupt?	ROA(C) before interest and depreciation before interest	ROA(A) before interest and % after tax	ROA(B) before interest and depreciation after tax	Operating Gross Margin	Realized Sales Gross Margin	Operating Profit Rate	Pre-tax net Interest Rate	After-tax net Interest Rate	Non-industry income and expenditure/revenue	...	Inc to T As
6814	0	0.493687	0.539468	0.543230	0.604455	0.604462	0.998992	0.797409	0.809331	0.303510	...	0.799
6815	0	0.475162	0.538269	0.524172	0.598308	0.598308	0.998992	0.797414	0.809327	0.303520	...	0.799
6816	0	0.472725	0.533744	0.520638	0.610444	0.610213	0.998984	0.797401	0.809317	0.303512	...	0.797
6817	0	0.506264	0.559911	0.554045	0.607850	0.607850	0.999074	0.797500	0.809399	0.303498	...	0.81
6818	0	0.493053	0.570105	0.549548	0.627409	0.627409	0.998080	0.801987	0.813800	0.313415	...	0.815

5 rows × 96 columns

In [19]:
```
df.isnull().sum()
```

Out[19]:
```
Bankrupt?                                                 0
 ROA(C) before interest and depreciation before interest    0
 ROA(A) before interest and % after tax                     0
 ROA(B) before interest and depreciation after tax          0
 Operating Gross Margin                                     0
                                                           ..
 Liability to Equity                                        0
 Degree of Financial Leverage (DFL)                         0
 Interest Coverage Ratio (Interest expense to EBIT)         0
 Net Income Flag                                            0
 Equity to Liability                                        0
Length: 96, dtype: int64
```

In [20]:
```
df.describe()
```

Out[20]:

	Bankrupt?	ROA(C) before interest and depreciation before interest	ROA(A) before interest and % after tax	ROA(B) before interest and depreciation after tax	Operating Gross Margin	Realized Sales Gross Margin	Operating Profit Rate	Pre-tax net Interest Rate	After-tax net Interest Rate	Nor in expenditure
count	6819.000000	6819.000000	6819.000000	6819.000000	6819.000000	6819.000000	6819.000000	6819.000000	6819.000000	68
mean	0.032263	0.505180	0.558625	0.553589	0.607948	0.607929	0.998755	0.797190	0.809084	
std	0.176710	0.060686	0.065620	0.061595	0.016934	0.016916	0.013010	0.012869	0.013601	
min	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000	
25%	0.000000	0.476527	0.535543	0.527277	0.600445	0.600434	0.998969	0.797386	0.809312	
50%	0.000000	0.502706	0.559802	0.552278	0.605997	0.605976	0.999022	0.797464	0.809375	
75%	0.000000	0.535563	0.589157	0.584105	0.613914	0.613842	0.999095	0.797579	0.809469	
max	1.000000	1.000000	1.000000	1.000000	1.000000	1.000000	1.000000	1.000000	1.000000	

8 rows × 96 columns

In [21]:
```
df['Bankrupt?'].value_counts()
```

Out[21]:
```
Bankrupt?
0    6599
1     220
Name: count, dtype: int64
```

In [22]:
```
df['Bankrupt?'].value_counts(normalize=True) * 100
```

Out[22]:
```
Bankrupt?
0    96.77372
1     3.22628
Name: proportion, dtype: float64
```

In [23]:
```
df['Bankrupt?'].value_counts()[0] / df['Bankrupt?'].value_counts()[1]
```

Out[23]: np.float64(29.995454545454546)

In [24]:
```
print("Total Records:", df.shape[0])
print("Total Features:", df.shape[1])
print("\nTarget Variable Distribution:")
print(df['Bankrupt?'].value_counts())
```

```
Total Records: 6819
Total Features: 96

Target Variable Distribution:
Bankrupt?
0    6599
1     220
Name: count, dtype: int64
```

Step 4 | Target Varible Analysis

In [25]:
```
target = 'Bankrupt?'
target in df.columns
```

Out[25]: True

In [26]:
```
df[target].value_counts()
```

Out[26]:
```
Bankrupt?
0    6599
1     220
Name: count, dtype: int64
```

In [27]:
```
df[target].value_counts(normalize=True) * 100
```

Out[27]:
```
Bankrupt?
0    96.77372
1     3.22628
Name: proportion, dtype: float64
```

In [28]:
```
non_bankrupt = df[target].value_counts()[0]
bankrupt = df[target].value_counts()[1]

imbalance_ratio = non_bankrupt / bankrupt
imbalance_ratio
```

Out[28]:
```
np.float64(29.995454545454546)
```

In [29]:
```
print("Non-Bankrupt Companies:", non_bankrupt)
print("Bankrupt Companies:", bankrupt)
print("Imbalance Ratio (Non-Bankrupt : Bankrupt):", round(imbalance_ratio, 2))
```

```
Non-Bankrupt Companies: 6599
Bankrupt Companies: 220
Imbalance Ratio (Non-Bankrupt : Bankrupt): 30.0
```

In [30]:
```
df[target].unique()
```

Out[30]:
```
array([1, 0])
```

In [31]:
```
df[target].dtype
```

Out[31]:
```
dtype('int64')
```

In [32]:
```
if imbalance_ratio > 1:
    print("The dataset is imbalanced, with significantly fewer bankrupt companies.")
else:
    print("The dataset is balanced.")
```

```
The dataset is imbalanced, with significantly fewer bankrupt companies.
```

Target Variable Analysis

The target variable `Bankrupt?` is a binary classification variable, where 1 represents bankrupt companies and 0 represents non-bankrupt companies. The analysis shows a significant class imbalance, with far fewer bankrupt cases compared to non-bankrupt cases. This imbalance reflects real-world financial scenarios and highlights the need for appropriate preprocessing techniques during model training.

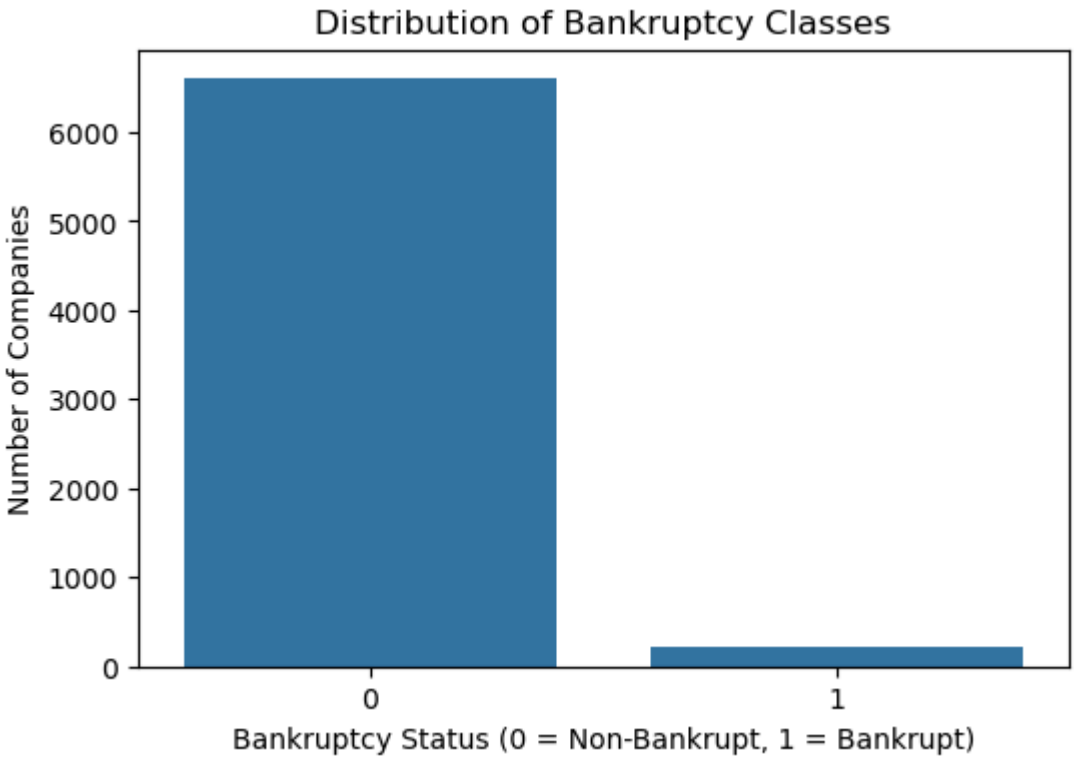
Class Imbalance Handling

The target variable shows a strong class imbalance, with significantly fewer bankrupt companies compared to non-bankrupt companies. This reflects real-world scenarios. Therefore, evaluation metrics such as Precision, Recall, and F1-score were prioritized over accuracy for model evaluation and comparison.

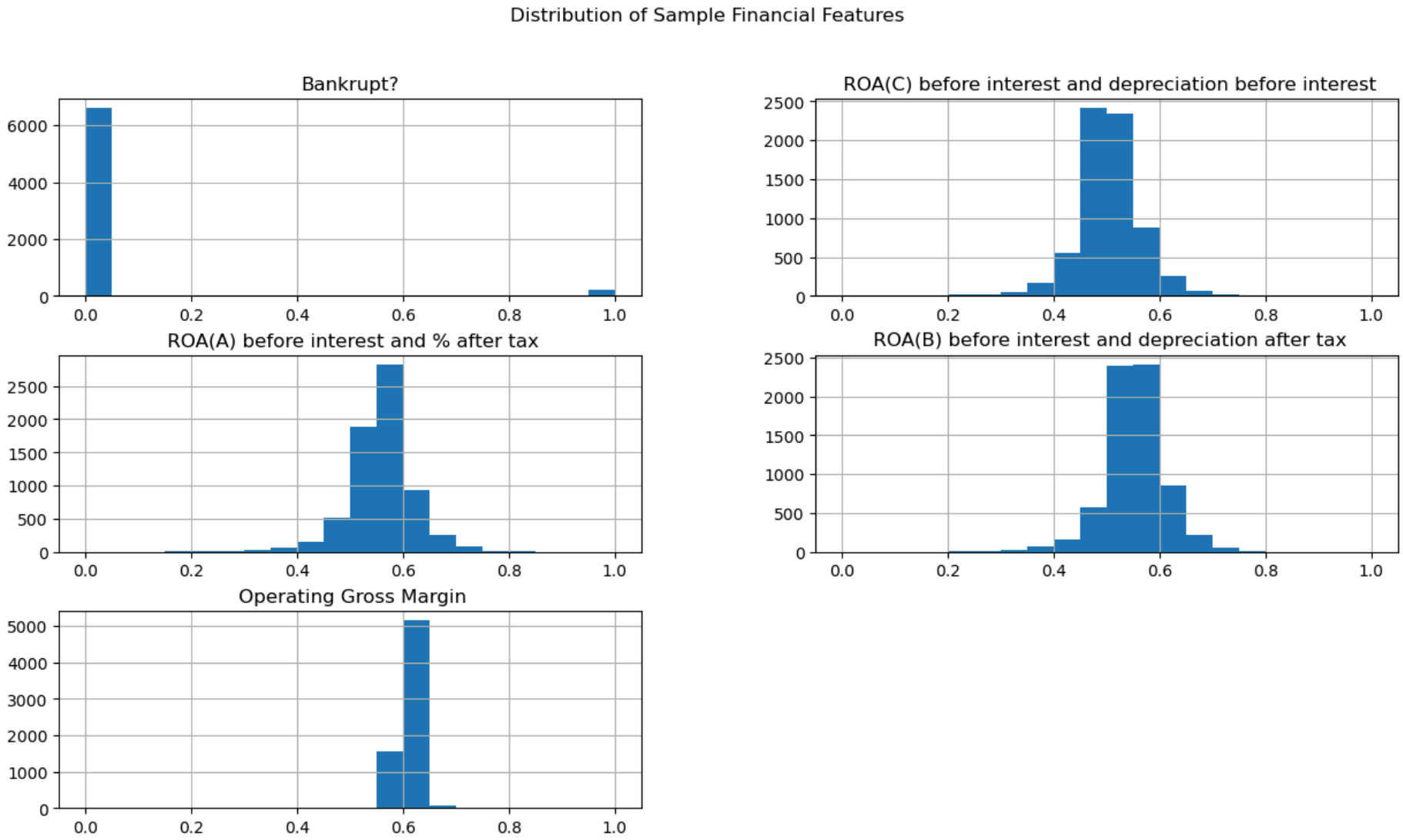
Step 5 | Exploratory Data Analysis (EDA)

In [33]:
```
import matplotlib.pyplot as plt
import seaborn as sns
import numpy as np
```

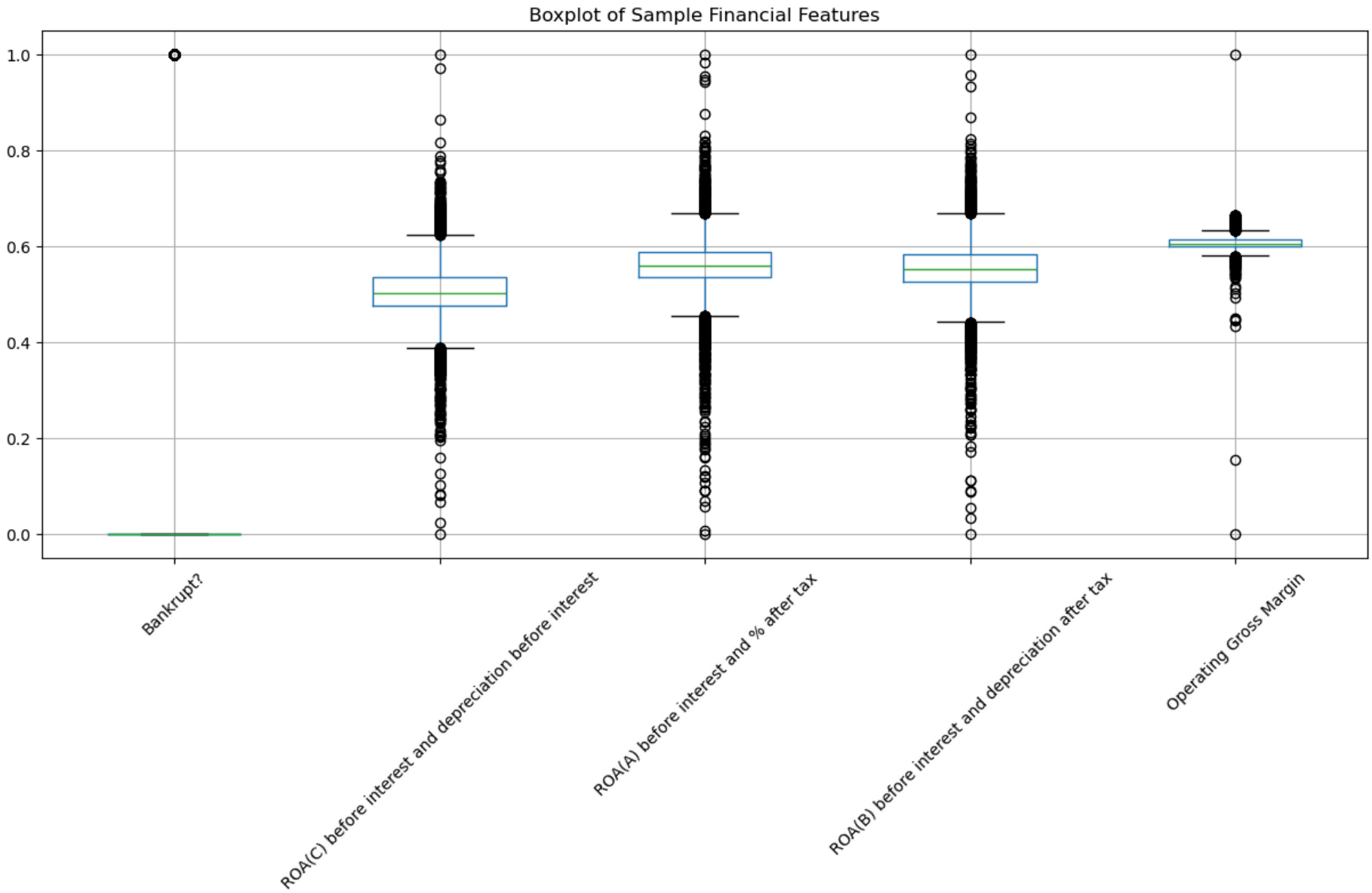
In [34]:
```
plt.figure(figsize=(6,4))
sns.countplot(x='Bankrupt?', data=df)
plt.title("Distribution of Bankruptcy Classes")
plt.xlabel("Bankruptcy Status (0 = Non-Bankrupt, 1 = Bankrupt)")
plt.ylabel("Number of Companies")
plt.show()
```



In [35]:
```
df.iloc[:, :5].hist(figsize=(15,8), bins=20)
plt.suptitle("Distribution of Sample Financial Features")
plt.show()
```



In [36]:
```
plt.figure(figsize=(15,6))
df.iloc[:, :5].boxplot()
plt.title("Boxplot of Sample Financial Features")
plt.xticks(rotation=45)
plt.show()
```



In [37]:
```
df.groupby('Bankrupt?').mean().head()
```

Out[37]:

Bankrupt?	ROA(C) before interest and depreciation before interest	ROA(A) before interest and % after tax	ROA(B) before interest and depreciation after tax	Operating Gross Margin	Realized Sales Gross Margin	Operating Profit Rate	Pre-tax net Interest Rate	After-tax net Interest Rate	Non-industry income and expenditure/revenue	Continuous interest rate (after tax)	...
0	0.508069	0.562015	0.556659	0.608257	0.608237	0.998756	0.79721	0.809106	0.303657	0.781401	...
1	0.418503	0.456947	0.461483	0.598670	0.598717	0.998739	0.79659	0.808424	0.302609	0.780799	...

2 rows × 95 columns

In [38]:

```
correlation_matrix = df.corr()
correlation_matrix.head()
```

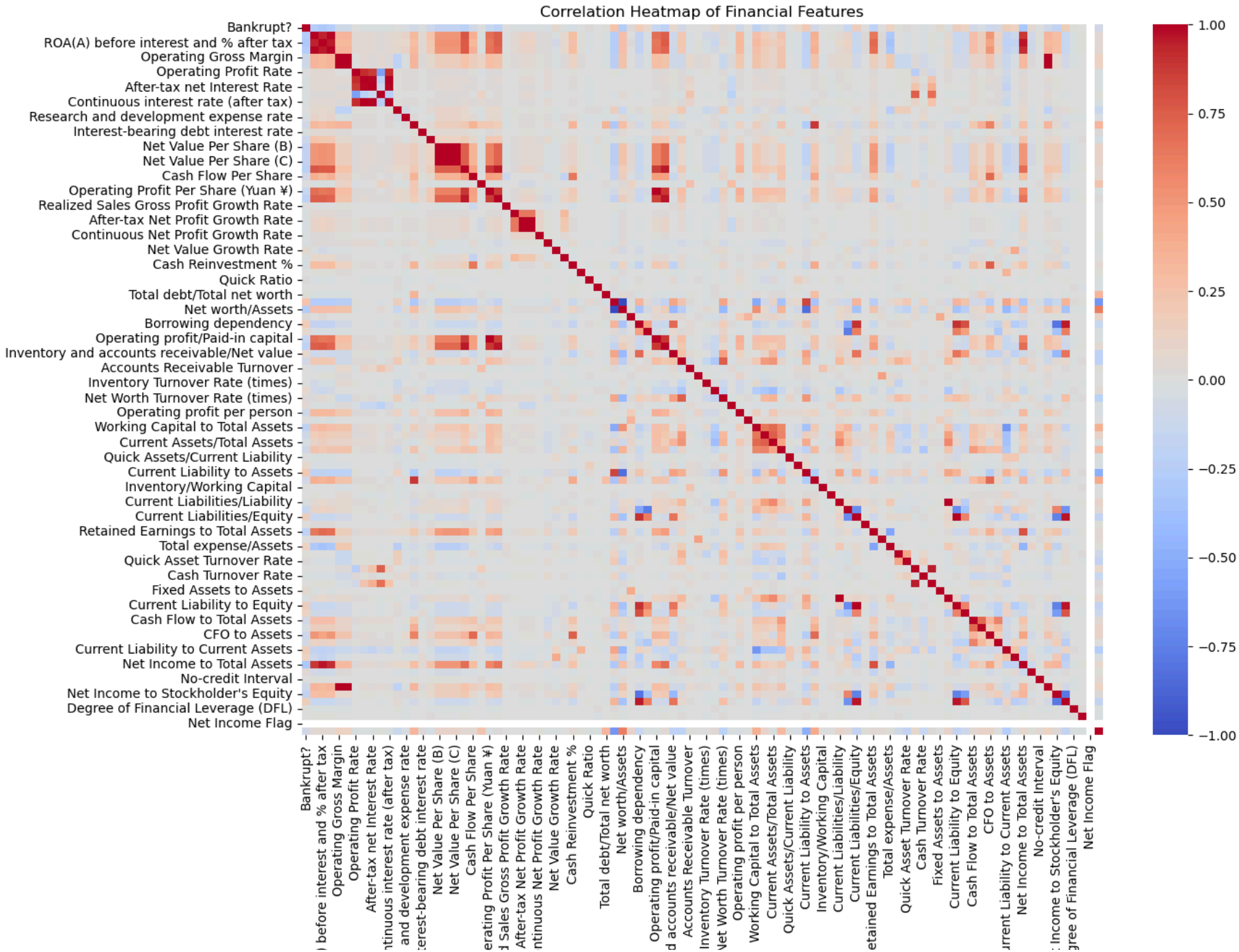
Out[38]:

	Bankrupt?	ROA(C) before interest and depreciation before interest	ROA(A) before interest and % after tax	ROA(B) before interest and depreciation after tax	Operating Gross Margin	Realized Sales Gross Margin	Operating Profit Rate	Pre-tax net Interest Rate	After-tax net Interest Rate	Non-industry income and expenditure/revenue
Bankrupt?	1.000000	-0.260807	-0.282941	-0.273051	-0.100043	-0.099445	-0.000230	-0.008517	-0.008857	-0.016593
ROA(C) before interest and depreciation before interest	-0.260807	1.000000	0.940124	0.986849	0.334719	0.332755	0.035725	0.053419	0.049222	0.020501
ROA(A) before interest and % after tax	-0.282941	0.940124	1.000000	0.955741	0.326969	0.324956	0.032053	0.053518	0.049474	0.029649
ROA(B) before interest and depreciation after tax	-0.273051	0.986849	0.955741	1.000000	0.333749	0.331755	0.035212	0.053726	0.049952	0.022366
Operating Gross Margin	-0.100043	0.334719	0.326969	0.333749	1.000000	0.999518	0.005745	0.032493	0.027175	0.051438

5 rows × 96 columns

In [39]:

```
plt.figure(figsize=(14,10))
sns.heatmap(correlation_matrix, cmap='coolwarm', center=0)
plt.title("Correlation Heatmap of Financial Features")
plt.show()
```


Correlation Heatmap of Financial Features
Bankrupt?
ROA(A) before interest and % after tax
Operating Gross Margin
Operating Profit Rate
After-tax net Interest Rate
Continuous interest rate (after tax)
Research and development expense rate
Interest-bearing debt interest rate
Net Value Per Share (B)
Net Value Per Share (C)
Cash Flow Per Share
Operating Profit Per Share (Yuan ¥)
Realized Sales Gross Profit Growth Rate
After-tax Net Profit Growth Rate
Continuous Net Profit Growth Rate
Net Value Growth Rate
Cash Reinvestment %
Quick Ratio
Total debt/Total net worth
Net worth/Assets
Borrowing dependency
Operating profit/Paid-in capital
Inventory and accounts receivable/Net value
Accounts Receivable Turnover
Inventory Turnover Rate (times)
Net Worth Turnover Rate (times)
Operating profit per person
Working Capital to Total Assets
Current Assets/Total Assets
Quick Assets/Current Liability
Current Liability to Assets
Inventory/Working Capital
Current Liabilities/Liability
Current Liabilities/Equity
Retained Earnings to Total Assets
Total expense/Assets
Quick Asset Turnover Rate
Cash Turnover Rate
Fixed Assets to Assets
Current Liability to Equity
Cash Flow to Total Assets
CFO to Assets
Current Liability to Current Assets
Net Income to Total Assets
No-credit Interval
Net Income to Stockholder's Equity
Degree of Financial Leverage (DFL)
Net Income Flag
1.00
0.75
0.50
0.25
0.00
−0.25
−0.50
−0.75
−1.00

In [40]:
```
# Correlation of all features with target variable
target_corr = correlation_matrix['Bankrupt?'].sort_values(ascending=False)
target_corr.head(15)
```

Out[40]:
```
Bankrupt?                                    1.000000
 Debt ratio %                                0.250161
 Current Liability to Assets                 0.194494
 Borrowing dependency                        0.176543
 Current Liability to Current Assets         0.171306
 Liability to Equity                         0.166812
 Current Liabilities/Equity                  0.153828
 Current Liability to Equity                 0.153828
 Liability-Assets Flag                       0.139212
 Total expense/Assets                        0.139049
 Equity to Long-term Liability               0.139014
 Cash/Current Liability                      0.077921
 Inventory and accounts receivable/Net value 0.075278
 Fixed Assets Turnover Frequency             0.072818
 Contingent liabilities/Net worth            0.070455
Name: Bankrupt?, dtype: float64
```

In [41]:
```
print("Top Positively Correlated Features:")
print(target_corr.head(10))

print("\nTop Negatively Correlated Features:")
print(target_corr.tail(10))
```

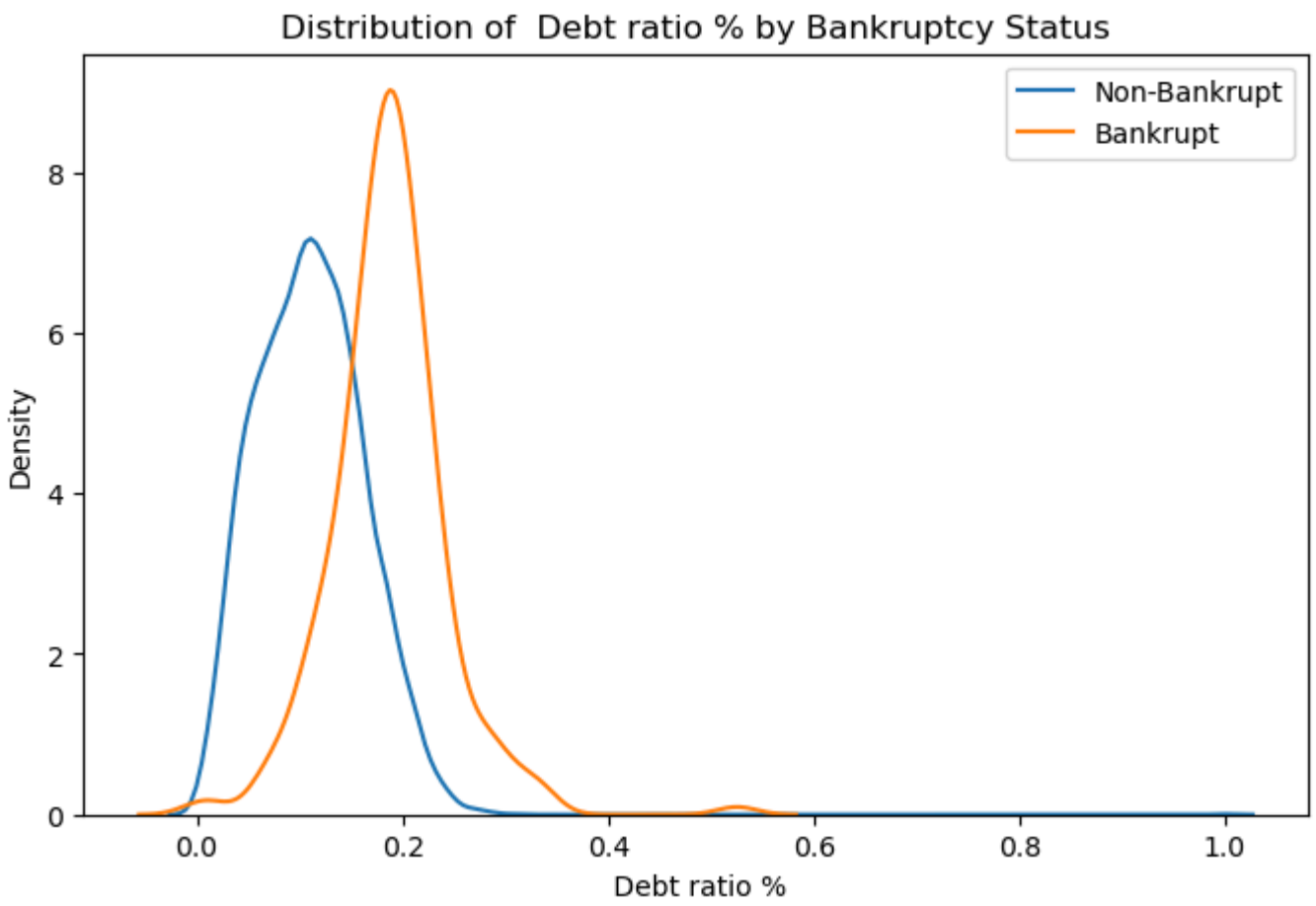
```
Top Positively Correlated Features:
Bankrupt?                             1.000000
 Debt ratio %                         0.250161
 Current Liability to Assets          0.194494
 Borrowing dependency                 0.176543
 Current Liability to Current Assets  0.171306
 Liability to Equity                  0.166812
 Current Liabilities/Equity           0.153828
 Current Liability to Equity          0.153828
 Liability-Assets Flag                0.139212
 Total expense/Assets                 0.139049
Name: Bankrupt?, dtype: float64

Top Negatively Correlated Features:
Per Share Net profit before tax (Yuan ¥)                 -0.201395
Net profit before tax/Paid-in capital                    -0.207857
Retained Earnings to Total Assets                        -0.217779
Persistent EPS in the Last Four Seasons                  -0.219560
Net worth/Assets                                         -0.250161
ROA(C) before interest and depreciation before interest  -0.260807
ROA(B) before interest and depreciation after tax        -0.273051
ROA(A) before interest and % after tax                   -0.282941
Net Income to Total Assets                               -0.315457
Net Income Flag                                                NaN
Name: Bankrupt?, dtype: float64
```

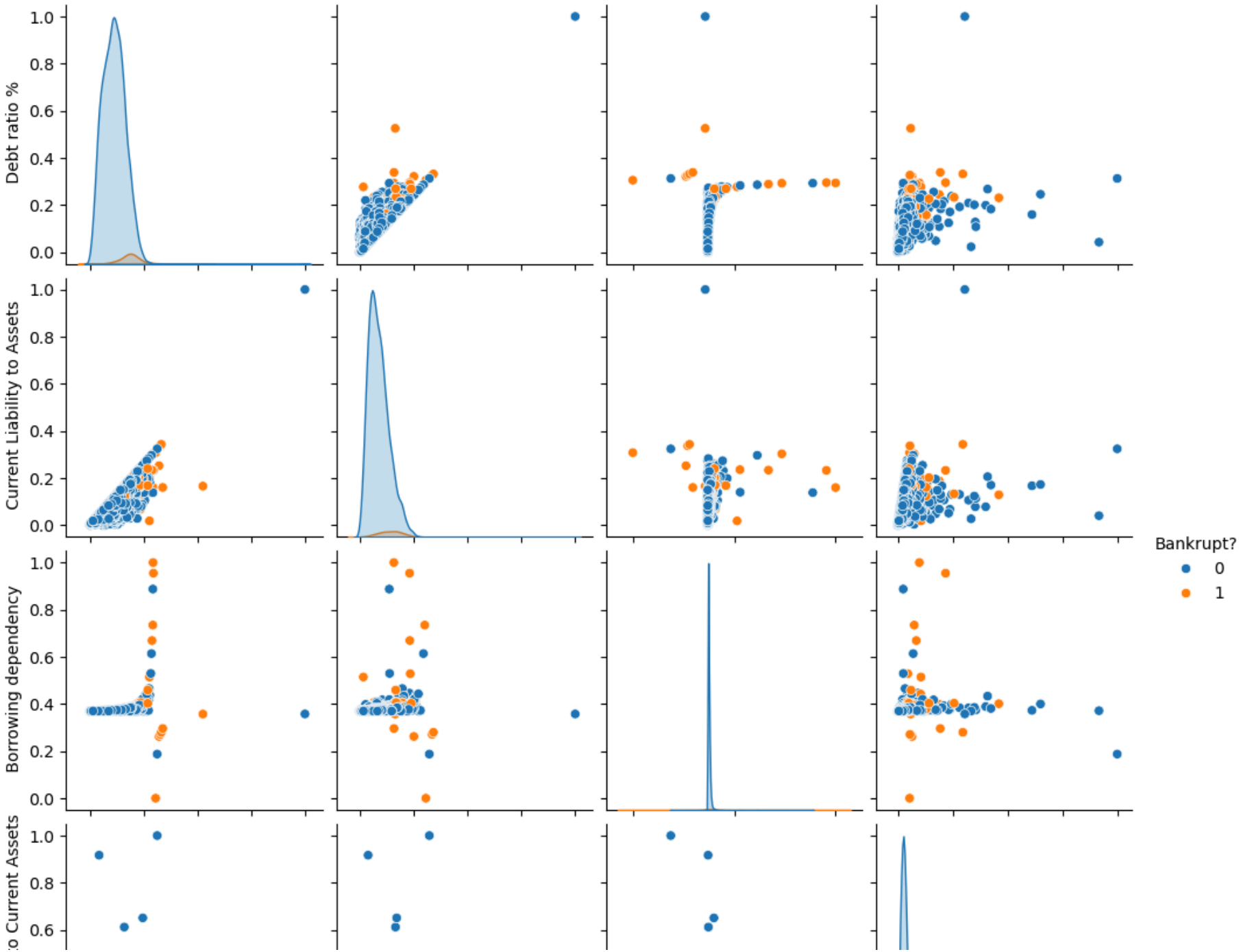
In [42]:

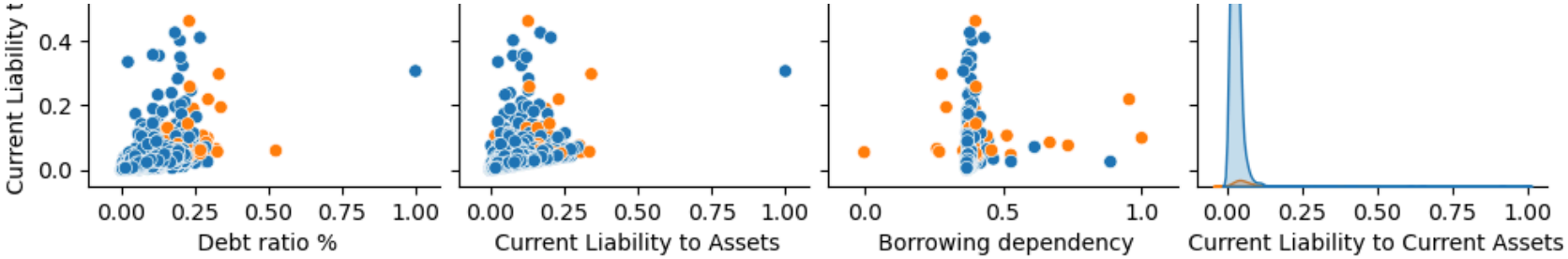
```
feature = target_corr.index[1]

plt.figure(figsize=(8,5))
sns.kdeplot(df[df['Bankrupt?'] == 0][feature], label='Non-Bankrupt')
sns.kdeplot(df[df['Bankrupt?'] == 1][feature], label='Bankrupt')
plt.title(f"Distribution of {feature} by Bankruptcy Status")
plt.legend()
plt.show()
```



In [43]:
```
sample_features = target_corr.index[1:5]
sns.pairplot(df[sample_features.tolist() + ['Bankrupt?']], hue='Bankrupt?')
plt.show()
```


Debt ratio %
Current Liability to Assets
Borrowing dependency
o Current Assets
1.0
0.8
0.6
0.4
0.2
0.0
Bankrupt?
0
1



Step 6 | Correlation Analysis & Insights

In [44]:
```
corr_matrix = df.corr()
```

In [45]:
```
df.corr()
```

Out[45]:

	Bankrupt?	ROA(C) before interest and depreciation before interest	ROA(A) before interest and % after tax	ROA(B) before interest and depreciation after tax	Operating Gross Margin	Realized Sales Gross Margin	Operating Profit Rate	Pre-tax net Interest Rate	After-tax net Interest Rate	Non-industry income and expenditure/revenue
Bankrupt?	1.000000	-0.260807	-0.282941	-0.273051	-0.100043	-0.099445	-0.000230	-0.008517	-0.008857	-0.016593
ROA(C) before interest and depreciation before interest	-0.260807	1.000000	0.940124	0.986849	0.334719	0.332755	0.035725	0.053419	0.049222	0.020501
ROA(A) before interest and % after tax	-0.282941	0.940124	1.000000	0.955741	0.326969	0.324956	0.032053	0.053518	0.049474	0.029649
ROA(B) before interest and depreciation after tax	-0.273051	0.986849	0.955741	1.000000	0.333749	0.331755	0.035212	0.053726	0.049952	0.022366
Operating Gross Margin	-0.100043	0.334719	0.326969	0.333749	1.000000	0.999518	0.005745	0.032493	0.027175	0.051438
...	...	...	...	...	...	...	...	...	...	...
Liability to Equity	0.166812	-0.143629	-0.141039	-0.142838	-0.085434	-0.085407	0.001541	-0.004043	-0.004390	-0.011899
Degree of Financial Leverage (DFL)	0.010508	-0.016575	-0.011515	-0.014663	-0.011806	-0.011268	0.000935	0.000855	0.000927	-0.000556

	Bankrupt?	ROA(C) before interest and depreciation before interest	ROA(A) before interest and % after tax	ROA(B) before interest and depreciation after tax	Operating Gross Margin	Realized Sales Gross Margin	Operating Profit Rate	Pre-tax net Interest Rate	After-tax net Interest Rate	Non-industry income and expenditure/revenue
Interest Coverage Ratio (Interest expense to EBIT)	-0.005509	0.010573	0.013372	0.011473	-0.001167	-0.001158	0.000393	0.000984	0.000957	0.001024
Net Income Flag	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN
Equity to Liability	-0.083048	0.052416	0.057887	0.056430	0.120029	0.120196	-0.017071	-0.014559	-0.010900	0.012293

96 rows × 96 columns

In [46]:
```
corr_matrix.head()
```

Out[46]:

	Bankrupt?	ROA(C) before interest and depreciation before interest	ROA(A) before interest and % after tax	ROA(B) before interest and depreciation after tax	Operating Gross Margin	Realized Sales Gross Margin	Operating Profit Rate	Pre-tax net Interest Rate	After-tax net Interest Rate	Non-industry income and expenditure/revenue
Bankrupt?	1.000000	-0.260807	-0.282941	-0.273051	-0.100043	-0.099445	-0.000230	-0.008517	-0.008857	-0.016593
ROA(C) before interest and depreciation before interest	-0.260807	1.000000	0.940124	0.986849	0.334719	0.332755	0.035725	0.053419	0.049222	0.020501
ROA(A) before interest and % after tax	-0.282941	0.940124	1.000000	0.955741	0.326969	0.324956	0.032053	0.053518	0.049474	0.029649
ROA(B) before interest and depreciation after tax	-0.273051	0.986849	0.955741	1.000000	0.333749	0.331755	0.035212	0.053726	0.049952	0.022366
Operating Gross Margin	-0.100043	0.334719	0.326969	0.333749	1.000000	0.999518	0.005745	0.032493	0.027175	0.051438

5 rows × 96 columns

In [47]:

```
target_corr = corr_matrix['Bankrupt?'].sort_values(ascending=False)
target_corr
```

Out[47]:
```
Bankrupt?                                                 1.000000
 Debt ratio %                                             0.250161
 Current Liability to Assets                              0.194494
 Borrowing dependency                                     0.176543
 Current Liability to Current Assets                      0.171306
                                                             ...
 ROA(C) before interest and depreciation before interest  -0.260807
 ROA(B) before interest and depreciation after tax        -0.273051
 ROA(A) before interest and % after tax                   -0.282941
 Net Income to Total Assets                               -0.315457
 Net Income Flag                                                NaN
Name: Bankrupt?, Length: 96, dtype: float64
```

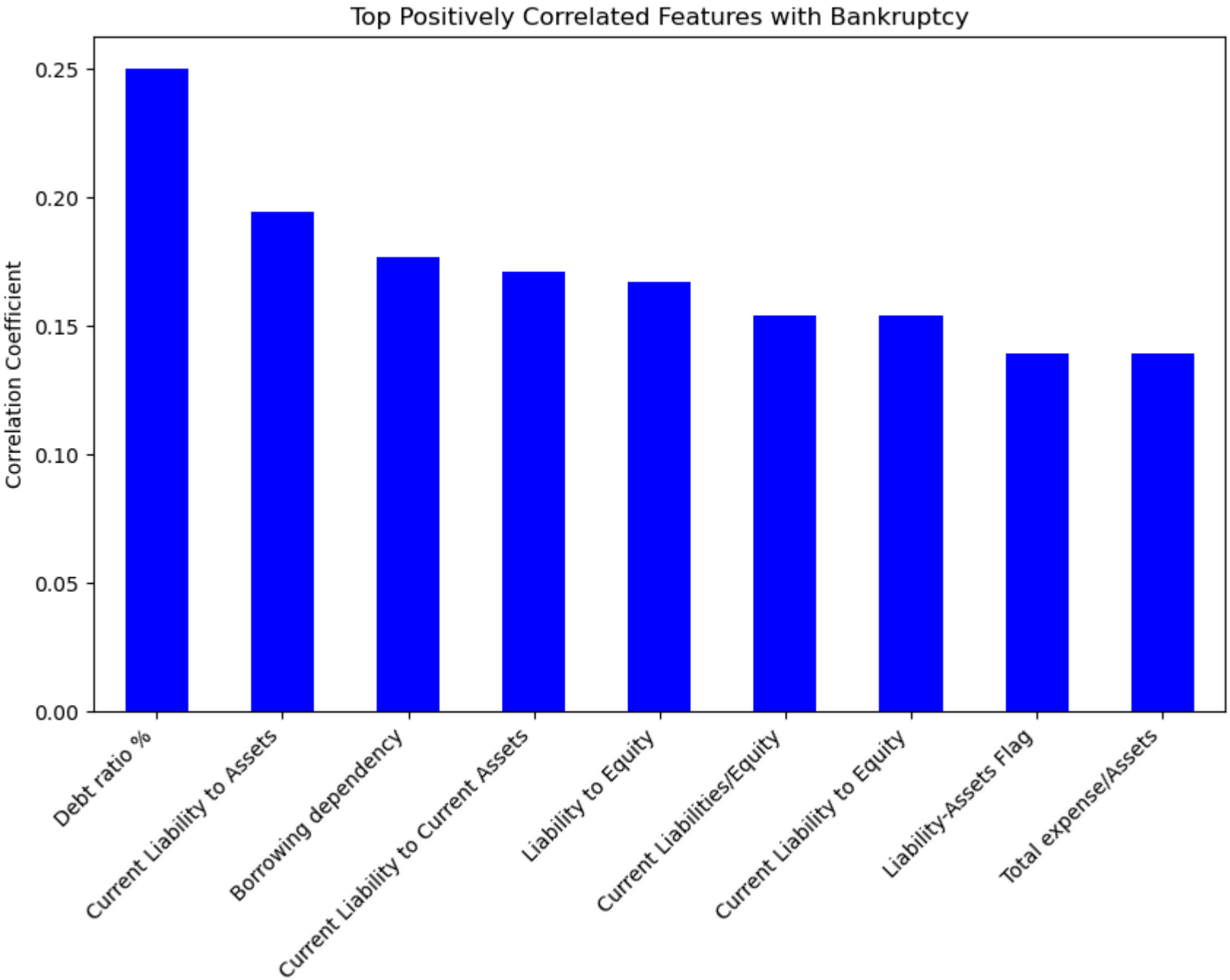
In [48]:
```
top_positive = target_corr.head(10)
top_positive
```

Out[48]:
```
Bankrupt?                             1.000000
 Debt ratio %                         0.250161
 Current Liability to Assets          0.194494
 Borrowing dependency                 0.176543
 Current Liability to Current Assets  0.171306
 Liability to Equity                  0.166812
 Current Liabilities/Equity           0.153828
 Current Liability to Equity          0.153828
 Liability-Assets Flag                0.139212
 Total expense/Assets                 0.139049
Name: Bankrupt?, dtype: float64
```

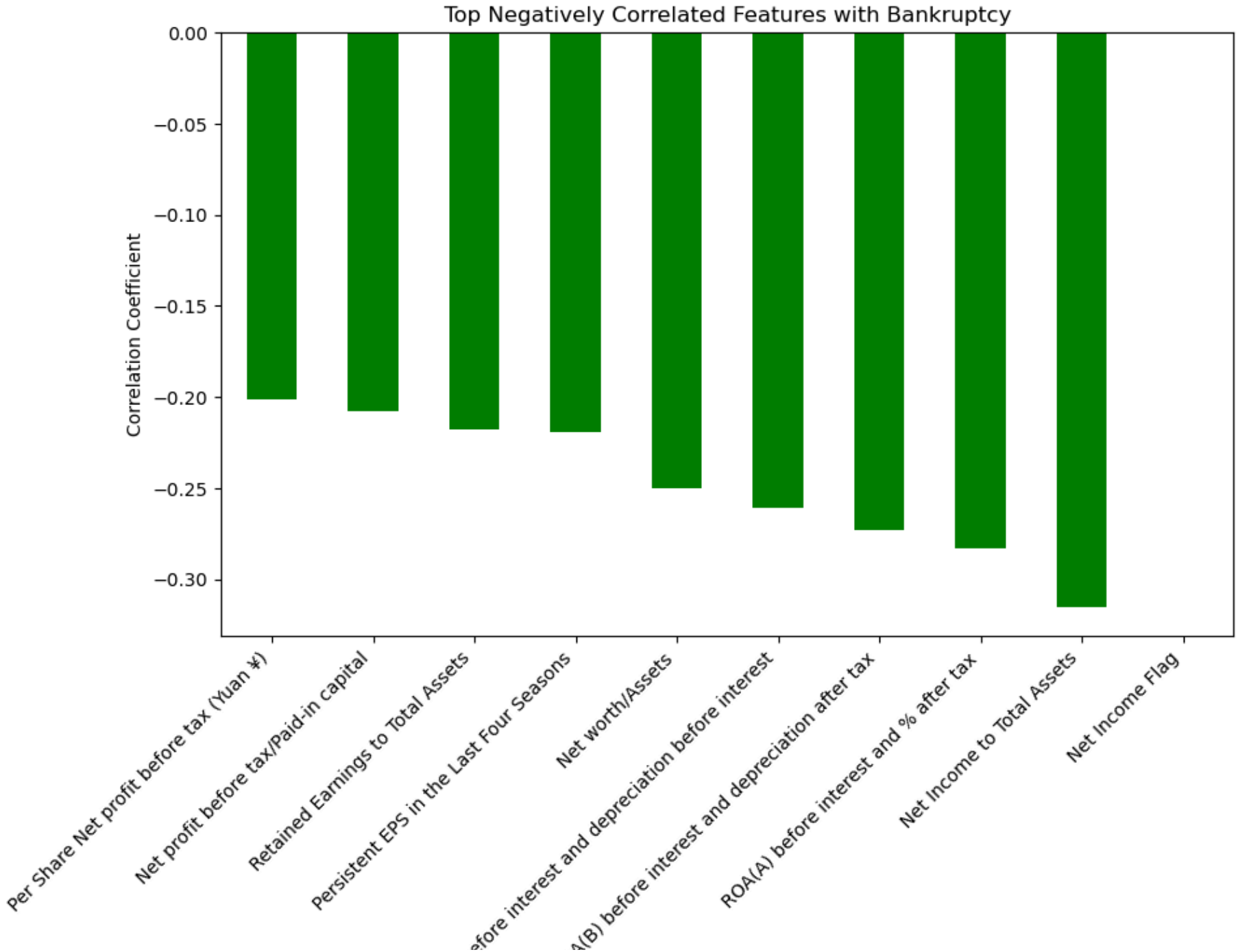
In [49]:
```
top_negative = target_corr.tail(10)
top_negative
```

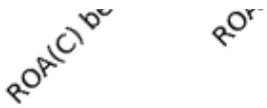
Out[49]:
```
Per Share Net profit before tax (Yuan ¥)                 -0.201395
Net profit before tax/Paid-in capital                    -0.207857
Retained Earnings to Total Assets                        -0.217779
Persistent EPS in the Last Four Seasons                  -0.219560
Net worth/Assets                                         -0.250161
ROA(C) before interest and depreciation before interest  -0.260807
ROA(B) before interest and depreciation after tax        -0.273051
ROA(A) before interest and % after tax                   -0.282941
Net Income to Total Assets                               -0.315457
Net Income Flag                                                NaN
Name: Bankrupt?, dtype: float64
```

In [50]:
```
plt.figure(figsize=(10,6))
top_positive[1:].plot(kind='bar', color='blue')
plt.title("Top Positively Correlated Features with Bankruptcy")
plt.ylabel("Correlation Coefficient")
plt.xticks(rotation=45, ha='right')
plt.show()
```


Top Positively Correlated Features with Bankruptcy
Correlation Coefficient
0.25
0.20
0.15
0.10
0.05
0.00
Debt ratio %
Current Liability to Assets
Borrowing dependency
Current Liability to Current Assets
Liability to Equity
Current Liabilities/Equity
Current Liability to Equity
Liability-Assets Flag
Total expense/Assets

In [51]:
```
plt.figure(figsize=(10,6))
top_negative.plot(kind='bar', color='green')
plt.title("Top Negatively Correlated Features with Bankruptcy")
plt.ylabel("Correlation Coefficient")
plt.xticks(rotation=45, ha='right')
plt.show()
```


Top Negatively Correlated Features with Bankruptcy
Correlation Coefficient
0.00
−0.05
−0.10
−0.15
−0.20
−0.25
−0.30
Per Share Net profit before tax (Yuan ¥)
Net profit before tax/Paid-in capital
Retained Earnings to Total Assets
Persistent EPS in the Last Four Seasons
Net worth/Assets
efore interest and depreciation before interest
A(B) before interest and depreciation after tax
ROA(A) before interest and % after tax
Net Income to Total Assets
Net Income Flag



In [52]:
```
threshold = 0.2

selected_features = target_corr[abs(target_corr) >= threshold].index
selected_features
```

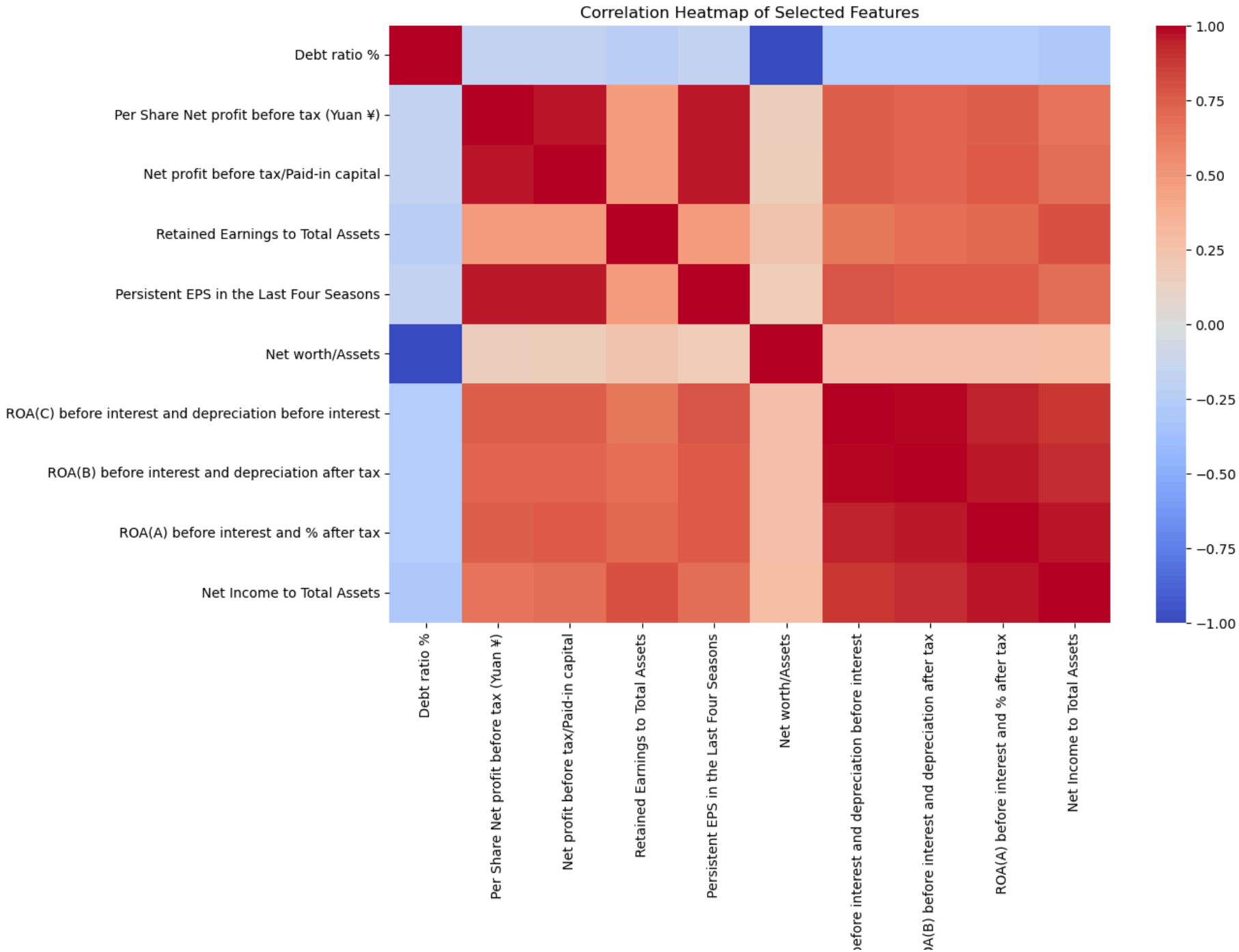
Out[52]:
```
Index(['Bankrupt?', ' Debt ratio %',
       ' Per Share Net profit before tax (Yuan ¥)',
       ' Net profit before tax/Paid-in capital',
       ' Retained Earnings to Total Assets',
       ' Persistent EPS in the Last Four Seasons', ' Net worth/Assets',
       ' ROA(C) before interest and depreciation before interest',
       ' ROA(B) before interest and depreciation after tax',
       ' ROA(A) before interest and % after tax',
       ' Net Income to Total Assets'],
      dtype='object')
```

In [53]:
```
X = df[selected_features].drop('Bankrupt?', axis=1)
y = df['Bankrupt?']

X.shape, y.shape
```

Out[53]:
```
((6819, 10), (6819,))
```

In [54]:
```
plt.figure(figsize=(12,8))
sns.heatmap(X.corr(), cmap='coolwarm', center=0)
plt.title("Correlation Heatmap of Selected Features")
plt.show()
```


Correlation Heatmap of Selected Features
Debt ratio %
Per Share Net profit before tax (Yuan ¥)
Net profit before tax/Paid-in capital
Retained Earnings to Total Assets
Persistent EPS in the Last Four Seasons
Net worth/Assets
ROA(C) before interest and depreciation before interest
ROA(B) before interest and depreciation after tax
ROA(A) before interest and % after tax
Net Income to Total Assets
1.00
0.75
0.50
0.25
0.00
−0.25
−0.50
−0.75
−1.00

Correlation Analysis & Insights

Correlation analysis was performed to understand the relationship between financial indicators and the bankruptcy status of companies. Several features showed strong positive and negative correlations with the target variable `Bankrupt?`, indicating their importance in predicting bankruptcy. Based on correlation strength, a subset of relevant features was selected to reduce dimensionality and improve model performance. This step helps in eliminating redundant variables and minimizing multicollinearity.

Step 7 | Feature Selection

In [55]:
```
X = df.drop('Bankrupt?', axis=1)
y = df['Bankrupt?']

X.shape, y.shape
```

Out[55]:
```
((6819, 95), (6819,))
```

In [56]:
```
corr_with_target = df.corr()['Bankrupt?'].abs().sort_values(ascending=False)
corr_with_target
```

Out[56]:
```
Bankrupt?                                                   1.000000
 Net Income to Total Assets                                 0.315457
 ROA(A) before interest and % after tax                     0.282941
 ROA(B) before interest and depreciation after tax          0.273051
 ROA(C) before interest and depreciation before interest    0.260807
                                                               ...
 Long-term Liability to Current Assets                      0.000778
 Cash Flow to Sales                                         0.000479
 Realized Sales Gross Profit Growth Rate                    0.000458
 Operating Profit Rate                                      0.000230
 Net Income Flag                                                 NaN
Name: Bankrupt?, Length: 96, dtype: float64
```

In [57]:
```
top_n = 15
selected_features_corr = corr_with_target[1:top_n+1].index.tolist()
selected_features_corr
```

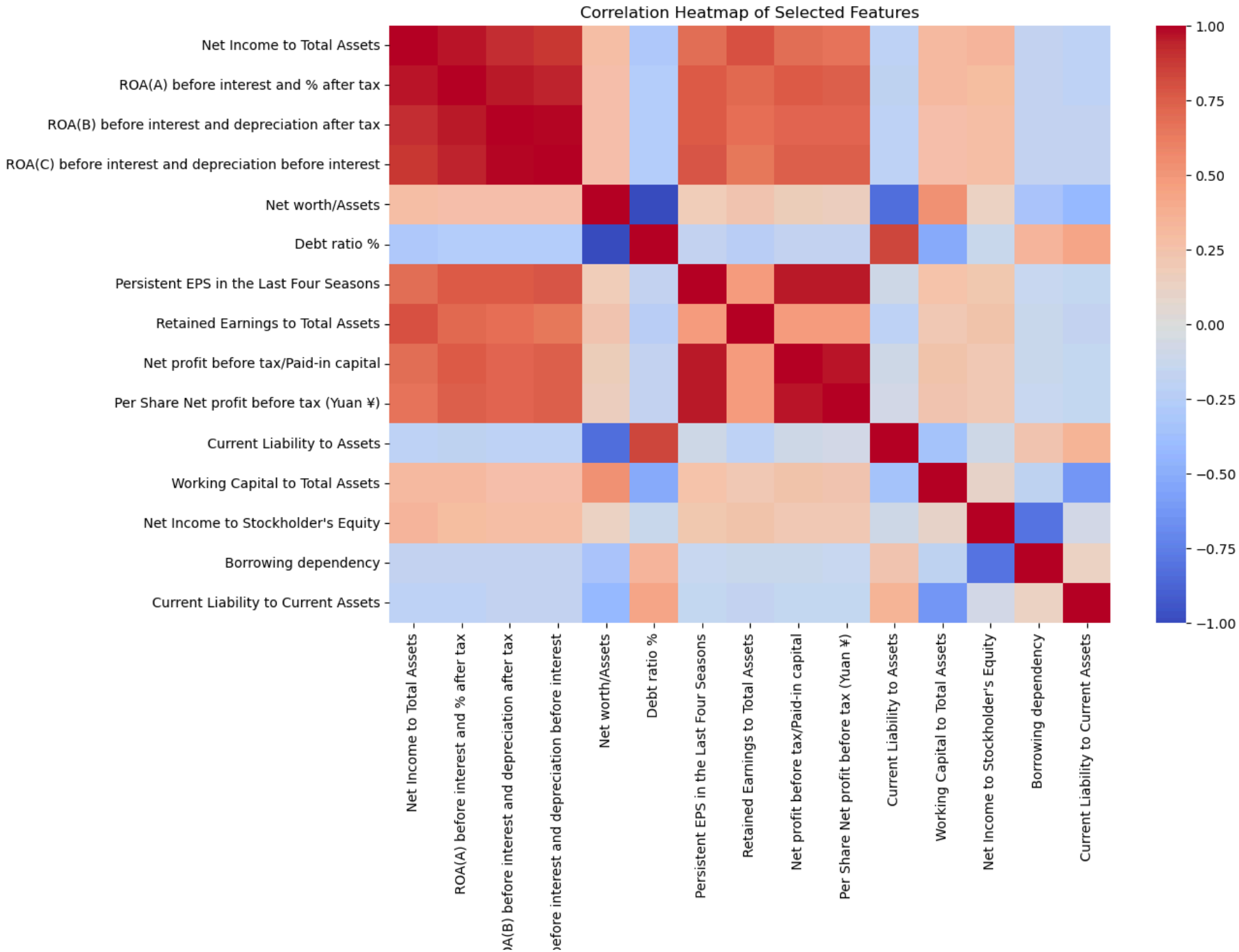
Out[57]:
```
[' Net Income to Total Assets',
 ' ROA(A) before interest and % after tax',
 ' ROA(B) before interest and depreciation after tax',
 ' ROA(C) before interest and depreciation before interest',
 ' Net worth/Assets',
 ' Debt ratio %',
 ' Persistent EPS in the Last Four Seasons',
 ' Retained Earnings to Total Assets',
 ' Net profit before tax/Paid-in capital',
 ' Per Share Net profit before tax (Yuan ¥)',
 ' Current Liability to Assets',
 ' Working Capital to Total Assets',
 " Net Income to Stockholder's Equity",
 ' Borrowing dependency',
 ' Current Liability to Current Assets']
```

In [58]:
```
X_corr = df[selected_features_corr]
X_corr.shape
```

Out[58]:
```
(6819, 15)
```

In [59]:
```
import seaborn as sns
import matplotlib.pyplot as plt
```

```
plt.figure(figsize=(12,8))
sns.heatmap(X_corr.corr(), cmap='coolwarm', center=0)
plt.title("Correlation Heatmap of Selected Features")
plt.show()
```


Correlation Heatmap of Selected Features
Net Income to Total Assets
ROA(A) before interest and % after tax
ROA(B) before interest and depreciation after tax
ROA(C) before interest and depreciation before interest
Net worth/Assets
Debt ratio %
Persistent EPS in the Last Four Seasons
Retained Earnings to Total Assets
Net profit before tax/Paid-in capital
Per Share Net profit before tax (Yuan ¥)
Current Liability to Assets
Working Capital to Total Assets
Net Income to Stockholder's Equity
Borrowing dependency
Current Liability to Current Assets
1.00
0.75
0.50
0.25
0.00
−0.25
−0.50
−0.75
−1.00

RC

ROA(C) b

In [60]:
```
from sklearn.feature_selection import VarianceThreshold
var_selector = VarianceThreshold(threshold=0.0001)
X_var = var_selector.fit_transform(X_corr)
X_var.shape
```

Out[60]:
```
(6819, 15)
```

In [61]:
```
selected_var_features = X_corr.columns[var_selector.get_support()]
selected_var_features
```

Out[61]:
```
Index([' Net Income to Total Assets',
       ' ROA(A) before interest and % after tax',
       ' ROA(B) before interest and depreciation after tax',
       ' ROA(C) before interest and depreciation before interest',
       ' Net worth/Assets', ' Debt ratio %',
       ' Persistent EPS in the Last Four Seasons',
       ' Retained Earnings to Total Assets',
       ' Net profit before tax/Paid-in capital',
       ' Per Share Net profit before tax (Yuan ¥)',
       ' Current Liability to Assets', ' Working Capital to Total Assets',
       ' Net Income to Stockholder's Equity', ' Borrowing dependency',
       ' Current Liability to Current Assets'],
      dtype='object')
```

In [62]:
```
from sklearn.feature_selection import SelectKBest, f_classif
selector = SelectKBest(score_func=f_classif, k=10)
X_univariate = selector.fit_transform(X_corr, y)
X_univariate.shape
```

Out[62]:
```
(6819, 10)
```

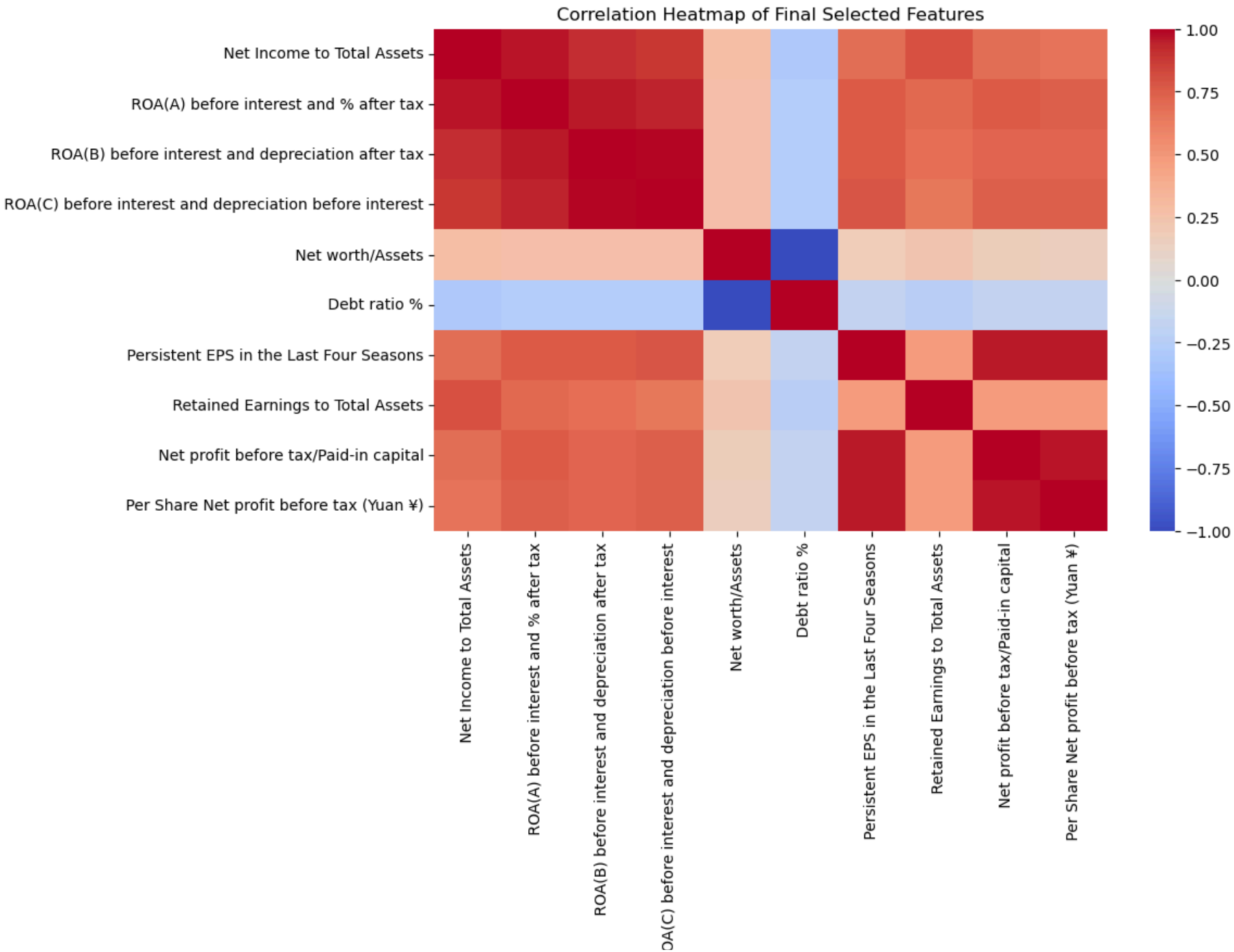
In [63]:
```
final_features = X_corr.columns[selector.get_support()]
final_features
```

Out[63]:
```
Index([' Net Income to Total Assets',
       ' ROA(A) before interest and % after tax',
       ' ROA(B) before interest and depreciation after tax',
       ' ROA(C) before interest and depreciation before interest',
       ' Net worth/Assets', ' Debt ratio %',
       ' Persistent EPS in the Last Four Seasons',
       ' Retained Earnings to Total Assets',
       ' Net profit before tax/Paid-in capital',
       ' Per Share Net profit before tax (Yuan ¥)'],
      dtype='object')
```

In [64]:
```
X_final = df[final_features]
X_final.shape
```

Out[64]:
```
(6819, 10)
```

In [65]:
```
plt.figure(figsize=(10,6))
sns.heatmap(X_final.corr(), cmap='coolwarm', center=0)
plt.title("Correlation Heatmap of Final Selected Features")
plt.show()
```


Correlation Heatmap of Final Selected Features
Net Income to Total Assets
ROA(A) before interest and % after tax
ROA(B) before interest and depreciation after tax
ROA(C) before interest and depreciation before interest
Net worth/Assets
Debt ratio %
Persistent EPS in the Last Four Seasons
Retained Earnings to Total Assets
Net profit before tax/Paid-in capital
Per Share Net profit before tax (Yuan ¥)
1.00
0.75
0.50
0.25
0.00
−0.25
−0.50
−0.75
−1.00

In [66]:
```
print("Initial number of features:", df.shape[1] - 1)
print("Features after correlation selection:", len(selected_features_corr))
print("Features after variance threshold:", len(selected_var_features))
print("Final selected features:", len(final_features))
```

```
Initial number of features: 95
Features after correlation selection: 15
Features after variance threshold: 15
Final selected features: 10
```

Feature Selection

Feature selection was performed to reduce dimensionality and improve model performance. Initially, features were selected based on their correlation with the target variable. Low-variance features were removed to eliminate non-informative variables. Finally, univariate feature selection using ANOVA F-test was applied to select the most significant financial indicators for bankruptcy prediction.

Evaluation Metrics

- Accuracy: Measures the overall correctness of the model.
- Precision: Indicates how many predicted bankrupt companies were actually bankrupt.
- Recall: Measures the model's ability to identify actual bankrupt companies.
- F1-Score: Harmonic mean of precision and recall, suitable for imbalanced datasets.

Step 8 | Data Preprocessing

In [67]:
```
X = df[final_features]
y = df['Bankrupt?']
X.shape, y.shape
```

Out[67]: ((6819, 10), (6819,))

In [68]:
```
from sklearn.model_selection import train_test_split
X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.2,
    random_state=42,
    stratify=y
)
X_train.shape, X_test.shape
```

Out[68]: ((5455, 10), (1364, 10))

In [69]:
```
from sklearn.preprocessing import StandardScaler
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)
X_train_scaled.shape, X_test_scaled.shape
```

Out[69]: ((5455, 10), (1364, 10))

In [70]:
```
print("Training set class distribution:")
print(y_train.value_counts())
print("\nTest set class distribution:")
print(y_test.value_counts())
```

```
Training set class distribution:
Bankrupt?
0    5279
1     176
Name: count, dtype: int64

Test set class distribution:
Bankrupt?
0    1320
1      44
Name: count, dtype: int64
```

In [71]:
```
from imblearn.over_sampling import SMOTE
smote = SMOTE(random_state=42)
X_train_balanced, y_train_balanced = smote.fit_resample(
    X_train_scaled,
    y_train
)
X_train_balanced.shape, y_train_balanced.shape
```

Out[71]:
```
((10558, 10), (10558,))
```

In [72]:
```
import pandas as pd
pd.Series(y_train_balanced).value_counts()
```

Out[72]:
```
Bankrupt?
0    5279
1    5279
Name: count, dtype: int64
```

In [73]:
```
print("Final Training Shape:", X_train_balanced.shape)
print("Final Test Shape:", X_test_scaled.shape)
```

```
Final Training Shape: (10558, 10)
Final Test Shape: (1364, 10)
```

Data Preprocessing

Data preprocessing was performed to prepare the dataset for machine learning models. The data was split into training and testing sets while maintaining class distribution. Feature scaling was applied using standardization to ensure uniform feature ranges. Due to significant class imbalance, SMOTE was used to balance the training data by generating synthetic samples for bankrupt companies. This preprocessing pipeline improves model stability and predictive performance.

Step 9| Feature Scaling & Train-Test Split

In [74]:
```
X = df[final_features]
y = df['Bankrupt?']
X.shape, y.shape
```

Out[74]:
```
((6819, 10), (6819,))
```

In [75]:
```
from sklearn.model_selection import train_test_split
X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.2,
    random_state=42,
    stratify=y
)

X_train.shape, X_test.shape
```

Out[75]:
```
((5455, 10), (1364, 10))
```

In [76]:
```
print("Training set distribution:")
print(y_train.value_counts())
print("\nTest set distribution:")
print(y_test.value_counts())
```

```
Training set distribution:
Bankrupt?
0    5279
1     176
Name: count, dtype: int64

Test set distribution:
Bankrupt?
0    1320
1      44
Name: count, dtype: int64
```

In [77]:
```
from sklearn.preprocessing import StandardScaler
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
```

```
X_test_scaled = scaler.transform(X_test)
X_train_scaled.shape, X_test_scaled.shape
```

Out[77]: ((5455, 10), (1364, 10))

In [78]:
```
X_train_scaled = pd.DataFrame(X_train_scaled, columns=final_features)
X_test_scaled = pd.DataFrame(X_test_scaled, columns=final_features)
```

In [79]:
```
X_train_scaled
```

Out[79]:

	Net Income to Total Assets	ROA(A) before interest and % after tax	ROA(B) before interest and depreciation after tax	ROA(C) before interest and depreciation before interest	Net worth/Assets	Debt ratio %	Persistent EPS in the Last Four Seasons	Retained Earnings to Total Assets	Net profit before tax/Paid-in capital	Per Share Net profit before tax (Yuan ¥)
0	-0.019743	-0.117303	-0.135577	-0.176677	1.083483	-1.083483	-0.216287	-0.060811	-0.237991	-0.214233
1	0.508205	0.504173	0.333551	0.496220	-0.870610	0.870610	0.649462	0.292229	0.791573	0.737985
2	0.530851	0.444945	0.221065	0.363741	0.899556	-0.899556	0.074272	0.285853	0.167204	0.166654
3	-0.239640	-0.320846	-0.519251	-0.575729	-1.059554	1.059554	-0.373427	0.024344	-0.372380	-0.370927
4	-1.272760	-1.753995	-1.962388	-1.870631	1.826998	-1.826998	-0.806301	-0.670633	-0.735815	-0.850651
...	...	...	...	...	...	...	...	...	...	...
5450	0.105601	0.092915	0.660546	0.756331	-0.683338	0.683338	0.352972	0.318291	0.252495	0.248617
5451	-0.388376	-0.454318	-0.373629	-0.378626	-0.805399	0.805399	-0.337848	-0.170742	-0.499178	-0.493871
5452	1.704199	2.293524	2.221401	1.952681	1.618546	-1.618546	2.760466	1.234898	2.290615	2.271175
5453	1.229745	1.290820	1.147114	1.365412	0.722316	-0.722316	0.631672	0.683869	0.765252	0.713878
5454	0.101183	0.134625	-0.044019	-0.057930	-0.394627	0.394627	-0.044323	0.128932	0.014845	0.009960

5455 rows × 10 columns

In [80]:
```
print("Scaled Training Data Shape:", X_train_scaled.shape)
print("Scaled Test Data Shape:", X_test_scaled.shape)
```

```
Scaled Training Data Shape: (5455, 10)
Scaled Test Data Shape: (1364, 10)
```

In [81]:
```
X_train_scaled
```

Out[81]:

	Net Income to Total Assets	ROA(A) before interest and % after tax	ROA(B) before interest and depreciation after tax	ROA(C) before interest and depreciation before interest	Net worth/Assets	Debt ratio %	Persistent EPS in the Last Four Seasons	Retained Earnings to Total Assets	Net profit before tax/Paid-in capital	Per Share Net profit before tax (Yuan ¥)
0	-0.019743	-0.117303	-0.135577	-0.176677	1.083483	-1.083483	-0.216287	-0.060811	-0.237991	-0.214233
1	0.508205	0.504173	0.333551	0.496220	-0.870610	0.870610	0.649462	0.292229	0.791573	0.737985
2	0.530851	0.444945	0.221065	0.363741	0.899556	-0.899556	0.074272	0.285853	0.167204	0.166654
3	-0.239640	-0.320846	-0.519251	-0.575729	-1.059554	1.059554	-0.373427	0.024344	-0.372380	-0.370927
4	-1.272760	-1.753995	-1.962388	-1.870631	1.826998	-1.826998	-0.806301	-0.670633	-0.735815	-0.850651
...	...	...	...	...	...	...	...	...	...	...
5450	0.105601	0.092915	0.660546	0.756331	-0.683338	0.683338	0.352972	0.318291	0.252495	0.248617
5451	-0.388376	-0.454318	-0.373629	-0.378626	-0.805399	0.805399	-0.337848	-0.170742	-0.499178	-0.493871
5452	1.704199	2.293524	2.221401	1.952681	1.618546	-1.618546	2.760466	1.234898	2.290615	2.271175
5453	1.229745	1.290820	1.147114	1.365412	0.722316	-0.722316	0.631672	0.683869	0.765252	0.713878
5454	0.101183	0.134625	-0.044019	-0.057930	-0.394627	0.394627	-0.044323	0.128932	0.014845	0.009960

5455 rows × 10 columns

In [82]:
```
X_test_scaled
```

Out[82]:

	Net Income to Total Assets	ROA(A) before interest and % after tax	ROA(B) before interest and depreciation after tax	ROA(C) before interest and depreciation before interest	Net worth/Assets	Debt ratio %	Persistent EPS in the Last Four Seasons	Retained Earnings to Total Assets	Net profit before tax/Paid-in capital	Per Share Net profit before tax (Yuan ¥)
0	-0.043884	-0.025541	-0.010883	-0.019964	-0.824907	0.824907	-0.216287	0.003676	-0.168138	-0.175662
1	-0.099412	-0.154007	-0.363165	-0.305925	0.120371	-0.120371	-0.248901	0.369667	-0.178768	-0.185304
2	0.796450	0.928779	0.922142	0.827417	0.093060	-0.093060	0.892583	0.427686	0.768795	0.807895
3	1.792658	2.058280	2.545779	2.642542	0.936898	-0.936898	1.601193	1.071570	1.588295	1.509401
4	-0.621281	-0.732105	-0.936932	-0.891579	1.783524	-1.783524	-0.631372	-0.240238	-0.531320	-0.525210
...	...	...	...	...	...	...	...	...	...	...
1359	0.242160	0.214707	-0.322182	-0.322080	-0.548458	0.548458	0.065378	0.147378	0.636936	0.610219
1360	-0.264125	-0.283308	-0.309974	-0.366509	-1.071259	1.071259	-0.411970	-0.108550	-0.467289	-0.469764
1361	0.429481	0.352350	0.178338	0.296694	-0.336105	0.336105	0.438954	0.385397	0.552910	0.537899
1362	-0.062049	-0.159847	-0.358805	-0.399629	-1.122536	1.122536	-0.136235	-0.007585	-0.211416	-0.149144
1363	-0.549685	-0.550250	-0.710216	-0.893194	-1.353839	1.353839	-0.797407	-0.220417	-0.808705	-0.872347

1364 rows × 10 columns

Train-Test Split and Feature Scaling

The dataset was divided into training and testing sets using a stratified split to preserve the original class distribution. Feature scaling was applied using standardization to bring all numerical features to a common scale. The scaler was fitted only on the training data to prevent data leakage and then applied to the test data.

Step 10| Logistic Regression Models

In [83]:
```
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import (
    accuracy_score,
    precision_score,
    recall_score,
    f1_score,
    confusion_matrix,
    classification_report
)
```

In [84]:
```
lr_model = LogisticRegression(
    max_iter=1000,
    random_state=42
)
lr_model.fit(X_train_scaled, y_train)
```

Out[84]:
```
▾ LogisticRegression
▶ Parameters
```

In [85]:
```
y_pred_lr = lr_model.predict(X_test_scaled)
```

In [86]:
```
y_pred_lr
```

Out[86]:
```
array([0, 0, 0, ..., 0, 0, 0], shape=(1364,))
```

In [87]:
```
accuracy = accuracy_score(y_test, y_pred_lr)
precision = precision_score(y_test, y_pred_lr)
recall = recall_score(y_test, y_pred_lr)
f1 = f1_score(y_test, y_pred_lr)
print("Logistic Regression Performance")
print("Accuracy :", accuracy)
```

```
print("Precision:", precision)
print("Recall   :", recall)
print("F1 Score :", f1)
```

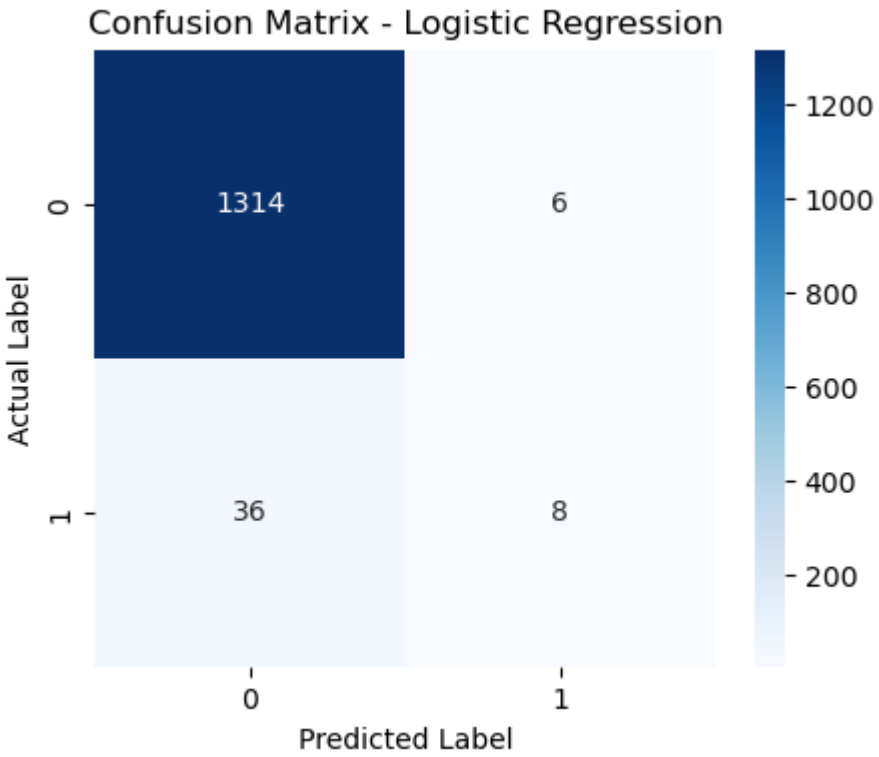
```
Logistic Regression Performance
Accuracy : 0.969208211143695
Precision: 0.5714285714285714
Recall   : 0.18181818181818182
F1 Score : 0.27586206896551724
```

In [88]:
```
cm = confusion_matrix(y_test, y_pred_lr)
```

In [89]:
```
cm
```

Out[89]:
```
array([[1314,    6],
       [  36,    8]])
```

In [90]:
```
import seaborn as sns
import matplotlib.pyplot as plt
plt.figure(figsize=(5,4))
sns.heatmap(
    cm,
    annot=True,
    fmt='d',
    cmap='Blues'
)
plt.xlabel("Predicted Label")
plt.ylabel("Actual Label")
plt.title("Confusion Matrix - Logistic Regression")
plt.show()
```



In [91]:
```
print(classification_report(y_test, y_pred_lr))
```

```
              precision    recall  f1-score   support

           0       0.97      1.00      0.98      1320
           1       0.57      0.18      0.28        44

    accuracy                           0.97      1364
   macro avg       0.77      0.59      0.63      1364
weighted avg       0.96      0.97      0.96      1364
```

In [92]:
```
coefficients = pd.DataFrame({
    'Feature': final_features,
    'Coefficient': lr_model.coef_[0]
}).sort_values(by='Coefficient', ascending=False)
coefficients
```

Out[92]:

	Feature	Coefficient
9	Per Share Net profit before tax (Yuan ¥)	0.899251
5	Debt ratio %	0.473481
7	Retained Earnings to Total Assets	0.279821
2	ROA(B) before interest and depreciation after...	0.154112
1	ROA(A) before interest and % after tax	0.087545
3	ROA(C) before interest and depreciation befor...	-0.422675
4	Net worth/Assets	-0.473481
8	Net profit before tax/Paid-in capital	-0.518188
0	Net Income to Total Assets	-0.578860
6	Persistent EPS in the Last Four Seasons	-1.071380

In [93]:
```
lr_weighted = LogisticRegression(
    max_iter=1000,
    class_weight='balanced',
    random_state=42
)
lr_weighted.fit(X_train_scaled, y_train)
y_pred_weighted = lr_weighted.predict(X_test_scaled)
print(classification_report(y_test, y_pred_weighted))
```

```
              precision    recall  f1-score   support

           0       0.99      0.88      0.93      1320
           1       0.19      0.84      0.30        44

    accuracy                           0.88      1364
   macro avg       0.59      0.86      0.62      1364
weighted avg       0.97      0.88      0.91      1364
```

Logistic Regression

Logistic Regression was implemented as a baseline classification model for bankruptcy prediction. The model was trained on scaled financial features and evaluated using accuracy, precision, recall, and F1-score. Due to class imbalance, a class-weighted version of Logistic Regression was also evaluated to improve minority class prediction performance.

Step 11| Decision Tree Model

In [94]:
```
from sklearn.tree import DecisionTreeClassifier
from sklearn.metrics import (
    accuracy_score,
    precision_score,
    recall_score,
    f1_score,
    confusion_matrix,
    classification_report
)
```

In [95]:
```
dt_model = DecisionTreeClassifier(
    random_state=42
)
dt_model.fit(X_train_scaled, y_train)
```

Out[95]:
```
▾ DecisionTreeClassifier
▶ Parameters
```

In [96]:
```
y_pred_dt = dt_model.predict(X_test_scaled)
```

In [97]:
```
y_pred_dt
```

Out[97]:
```
array([0, 0, 0, ..., 0, 0, 0], shape=(1364,))
```

In [98]:
```
accuracy_dt = accuracy_score(y_test, y_pred_dt)
precision_dt = precision_score(y_test, y_pred_dt)
recall_dt = recall_score(y_test, y_pred_dt)
f1_dt = f1_score(y_test, y_pred_dt)
print("Decision Tree Performance")
print("Accuracy :", accuracy_dt)
print("Precision:", precision_dt)
print("Recall   :", recall_dt)
print("F1 Score :", f1_dt)
```

```
Decision Tree Performance
Accuracy : 0.9633431085043989
Precision: 0.4318181818181818
Recall   : 0.4318181818181818
F1 Score : 0.4318181818181818
```

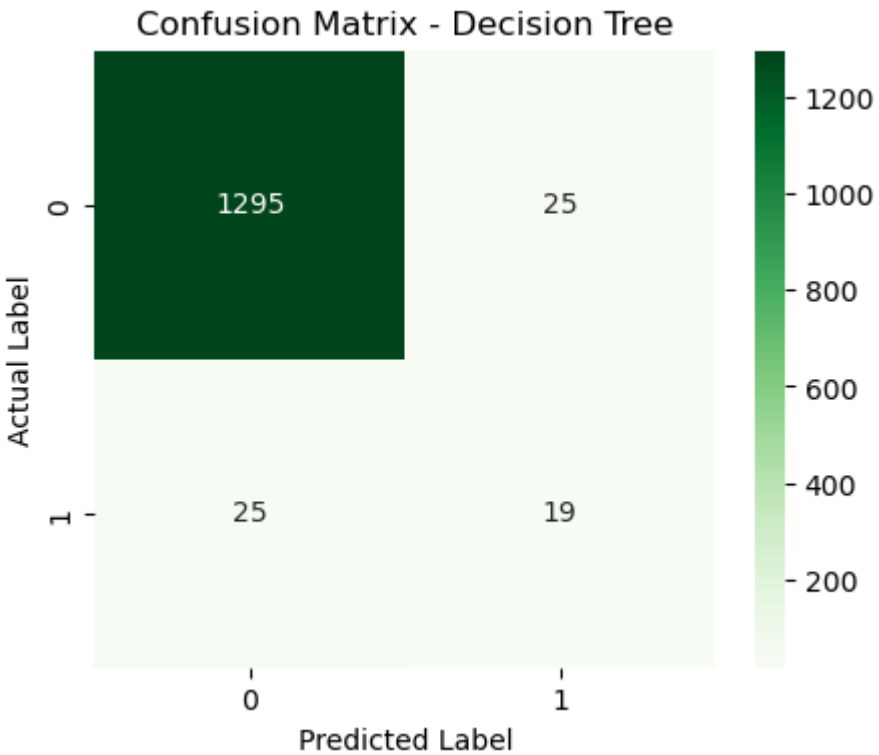
In [99]:
```
cm_dt = confusion_matrix(y_test, y_pred_dt)
cm_dt
```

Out[99]:
```
array([[1295,   25],
       [  25,   19]])
```

In [100…
```
import seaborn as sns
import matplotlib.pyplot as plt

plt.figure(figsize=(5,4))
sns.heatmap(
    cm_dt,
    annot=True,
```

```
    fmt='d',
    cmap='Greens'
)
plt.xlabel("Predicted Label")
plt.ylabel("Actual Label")
plt.title("Confusion Matrix - Decision Tree")
plt.show()
```



In [101…

```
print(classification_report(y_test, y_pred_dt))
```

```
              precision    recall  f1-score   support

           0       0.98      0.98      0.98      1320
           1       0.43      0.43      0.43        44

    accuracy                           0.96      1364
   macro avg       0.71      0.71      0.71      1364
weighted avg       0.96      0.96      0.96      1364
```

In [102…

```
feature_importance_dt = pd.DataFrame({
    'Feature': final_features,
    'Importance': dt_model.feature_importances_
}).sort_values(by='Importance', ascending=False)

feature_importance_dt
```

Out[102…

	Feature	Importance
6	Persistent EPS in the Last Four Seasons	0.266363
5	Debt ratio %	0.137791
2	ROA(B) before interest and depreciation after...	0.123779
9	Per Share Net profit before tax (Yuan ¥)	0.088168
8	Net profit before tax/Paid-in capital	0.081802
3	ROA(C) before interest and depreciation befor...	0.080756
1	ROA(A) before interest and % after tax	0.072722
0	Net Income to Total Assets	0.068478
7	Retained Earnings to Total Assets	0.059277
4	Net worth/Assets	0.020865

In [103…

```
dt_pruned = DecisionTreeClassifier(
    max_depth=5,
    min_samples_split=10,
```

```
    random_state=42
)

dt_pruned.fit(X_train_scaled, y_train)

y_pred_dt_pruned = dt_pruned.predict(X_test_scaled)

print(classification_report(y_test, y_pred_dt_pruned))
```

```
              precision    recall  f1-score   support

           0       0.97      1.00      0.99      1320
           1       0.75      0.14      0.23        44

    accuracy                           0.97      1364
   macro avg       0.86      0.57      0.61      1364
weighted avg       0.96      0.97      0.96      1364
```

Decision Tree

The Decision Tree classifier was used to model non-linear relationships among financial indicators. While the basic Decision Tree model can capture complex patterns, it is prone to overfitting. To address this, a pruned Decision Tree with controlled depth and minimum samples per split was implemented, resulting in improved generalization performance.

Step 12| Random Forest Model

In [104…

```
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import (
    accuracy_score,
    precision_score,
    recall_score,
    f1_score,
    confusion_matrix,
```

```
    classification_report
)
```

In [105…

```
rf_model = RandomForestClassifier(
    n_estimators=200,
    random_state=42,
    n_jobs=-1
)
rf_model.fit(X_train_scaled, y_train)
```

Out[105…

```
▾ RandomForestClassifier
▶ Parameters
```

In [106…

```
y_pred_rf = rf_model.predict(X_test_scaled)
```

In [107…

```
y_pred_rf
```

Out[107…

```
array([0, 0, 0, ..., 0, 0, 0], shape=(1364,))
```

In [108…

```
print("Random Forest Performance")
print("Accuracy :", accuracy_score(y_test, y_pred_rf))
print("Precision:", precision_score(y_test, y_pred_rf))
print("Recall   :", recall_score(y_test, y_pred_rf))
print("F1 Score :", f1_score(y_test, y_pred_rf))
```

```
Random Forest Performance
Accuracy : 0.9699413489736071
Precision: 0.6
Recall   : 0.20454545454545456
F1 Score : 0.3050847457627119
```

In [109…

```
cm_rf = confusion_matrix(y_test, y_pred_rf)
cm_rf
```

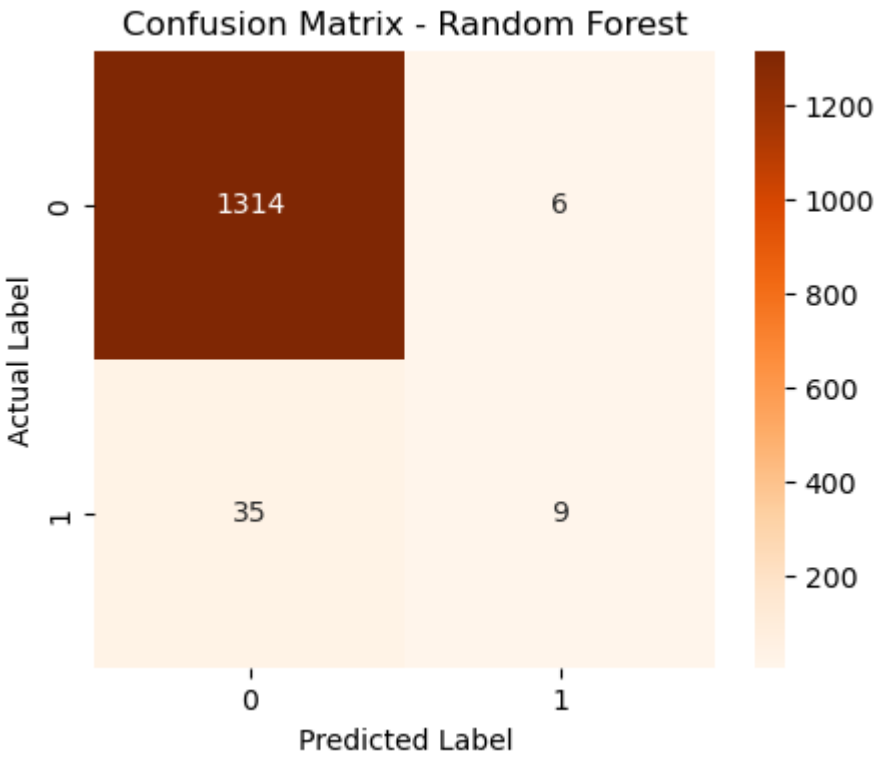
Out[109…

```
array([[1314,    6],
       [  35,    9]])
```

In [110…

```
import seaborn as sns
import matplotlib.pyplot as plt

plt.figure(figsize=(5,4))
sns.heatmap(cm_rf, annot=True, fmt='d', cmap='Oranges')
plt.xlabel("Predicted Label")
plt.ylabel("Actual Label")
plt.title("Confusion Matrix - Random Forest")
plt.show()
```



In [111…

```
rf_importance = pd.DataFrame({
    'Feature': final_features,
    'Importance': rf_model.feature_importances_
}).sort_values(by='Importance', ascending=False)

rf_importance
```

Out[111…

	Feature	Importance
6	Persistent EPS in the Last Four Seasons	0.134418
8	Net profit before tax/Paid-in capital	0.119926
5	Debt ratio %	0.113133
4	Net worth/Assets	0.111647
9	Per Share Net profit before tax (Yuan ¥)	0.109657
0	Net Income to Total Assets	0.096559
1	ROA(A) before interest and % after tax	0.089050
7	Retained Earnings to Total Assets	0.082513
3	ROA(C) before interest and depreciation befor...	0.072908
2	ROA(B) before interest and depreciation after...	0.070191

In [112…

```
from sklearn.ensemble import GradientBoostingClassifier
```

In [113…

```
gb_model = GradientBoostingClassifier(
    n_estimators=150,
    learning_rate=0.1,
    random_state=42
)
gb_model.fit(X_train_scaled, y_train)
```

Out[113…

```
▾ GradientBoostingClassifier

▶ Parameters
```

In [114…

```
y_pred_gb = gb_model.predict(X_test_scaled)
```

In [115…

```
print("Gradient Boosting Performance")
print("Accuracy :", accuracy_score(y_test, y_pred_gb))
```

```
print("Precision:", precision_score(y_test, y_pred_gb))
print("Recall   :", recall_score(y_test, y_pred_gb))
print("F1 Score :", f1_score(y_test, y_pred_gb))
```

```
Gradient Boosting Performance
Accuracy : 0.967741935483871
Precision: 0.5
Recall   : 0.20454545454545456
F1 Score : 0.2903225806451613
```

In [116…

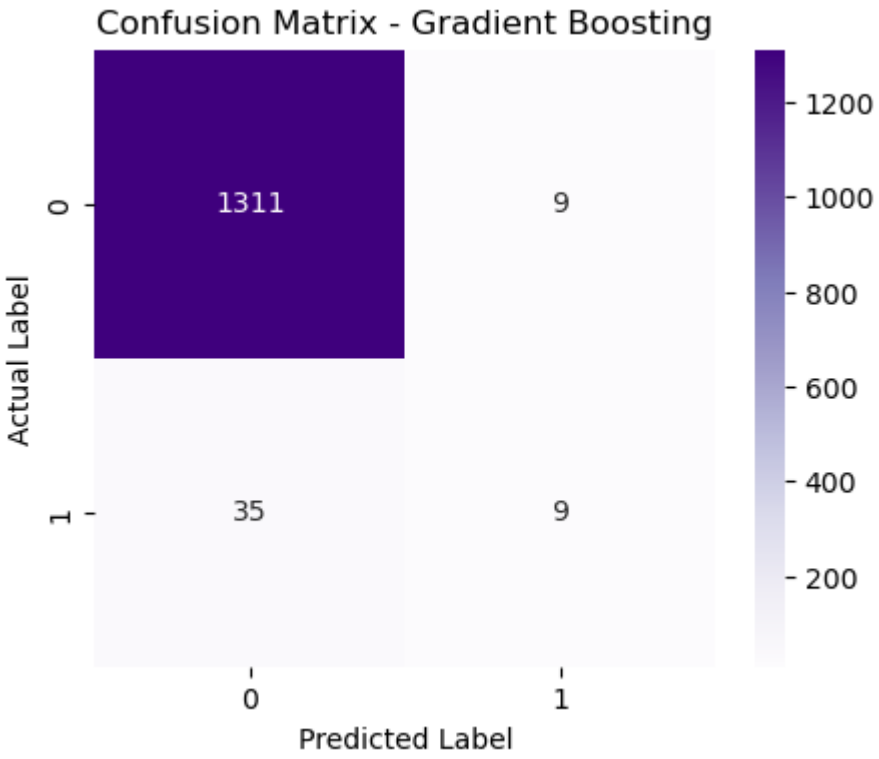
```
cm_gb = confusion_matrix(y_test, y_pred_gb)
cm_gb
```

Out[116…

```
array([[1311,    9],
       [  35,    9]])
```

In [117…

```
plt.figure(figsize=(5,4))
sns.heatmap(cm_gb, annot=True, fmt='d', cmap='Purples')
plt.xlabel("Predicted Label")
plt.ylabel("Actual Label")
plt.title("Confusion Matrix - Gradient Boosting")
plt.show()
```



Step 13| Gradient Boosting Models

In [118…

```
from sklearn.ensemble import GradientBoostingClassifier
from sklearn.metrics import (
    accuracy_score,
    precision_score,
    recall_score,
    f1_score,
    confusion_matrix,
    classification_report
)
```

```
import matplotlib.pyplot as plt
import seaborn as sns
```

In [119…

```
gb_model = GradientBoostingClassifier(
    n_estimators=150,
    learning_rate=0.1,
    max_depth=3,
    random_state=42
)
```

In [120…

```
gb_model
```

Out[120…

```
▾ GradientBoostingClassifier
▶ Parameters
```

In [121…

```
gb_model.fit(X_train_scaled, y_train)
```

Out[121…

```
▾ GradientBoostingClassifier
▶ Parameters
```

In [122…

```
accuracy_gb = accuracy_score(y_test, y_pred_gb)
precision_gb = precision_score(y_test, y_pred_gb)
recall_gb = recall_score(y_test, y_pred_gb)
f1_gb = f1_score(y_test, y_pred_gb)

print("Gradient Boosting Performance")
print("Accuracy :", accuracy_gb)
print("Precision:", precision_gb)
print("Recall   :", recall_gb)
print("F1 Score :", f1_gb)
```

```
Gradient Boosting Performance
Accuracy : 0.967741935483871
Precision: 0.5
Recall   : 0.20454545454545456
F1 Score : 0.2903225806451613
```

In [123…

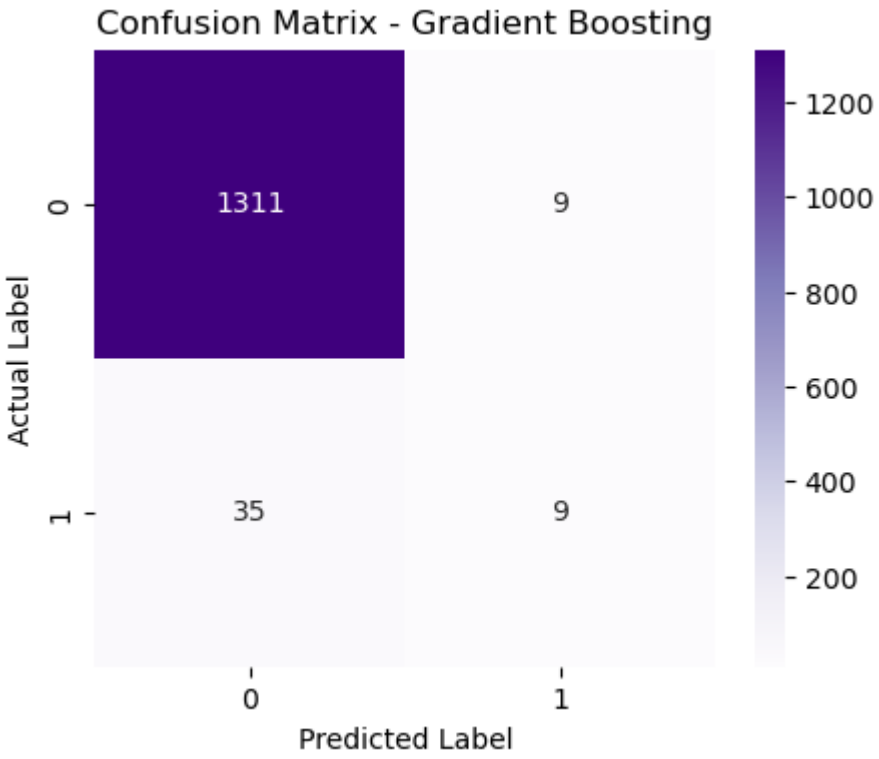
```
cm_gb = confusion_matrix(y_test, y_pred_gb)
cm_gb
```

Out[123…

```
array([[1311,    9],
       [  35,    9]])
```

In [124…

```
plt.figure(figsize=(5,4))
sns.heatmap(
    cm_gb,
    annot=True,
    fmt='d',
    cmap='Purples'
)
plt.xlabel("Predicted Label")
plt.ylabel("Actual Label")
plt.title("Confusion Matrix - Gradient Boosting")
plt.show()
```



In [125…

```
print(classification_report(y_test, y_pred_gb))
```

```
              precision    recall  f1-score   support

           0       0.97      0.99      0.98      1320
           1       0.50      0.20      0.29        44

    accuracy                           0.97      1364
   macro avg       0.74      0.60      0.64      1364
weighted avg       0.96      0.97      0.96      1364
```

In [126…

```
gb_importance = pd.DataFrame({
    'Feature': final_features,
    'Importance': gb_model.feature_importances_
}).sort_values(by='Importance', ascending=False)
```

```
gb_importance
```

Out[126…

	Feature	Importance
6	Persistent EPS in the Last Four Seasons	0.312150
5	Debt ratio %	0.134938
4	Net worth/Assets	0.118550
7	Retained Earnings to Total Assets	0.081883
9	Per Share Net profit before tax (Yuan ¥)	0.079531
0	Net Income to Total Assets	0.074285
1	ROA(A) before interest and % after tax	0.060082
8	Net profit before tax/Paid-in capital	0.054375
3	ROA(C) before interest and depreciation befor...	0.044807
2	ROA(B) before interest and depreciation after...	0.039398

In [127…

```
from sklearn.model_selection import GridSearchCV
param_grid = {
    'n_estimators': [100, 150, 200],
    'learning_rate': [0.05, 0.1],
    'max_depth': [3, 4]
}
grid_gb = GridSearchCV(
    GradientBoostingClassifier(random_state=42),
    param_grid,
    cv=5,
    scoring='f1',
    n_jobs=-1
)
grid_gb.fit(X_train_scaled, y_train)
grid_gb.best_params_
```

Out[127…
```
{'learning_rate': 0.1, 'max_depth': 3, 'n_estimators': 200}
```

In [128…
```
best_gb = grid_gb.best_estimator_
y_pred_best_gb = best_gb.predict(X_test_scaled)
print(classification_report(y_test, y_pred_best_gb))
```

```
              precision    recall  f1-score   support

           0       0.97      0.99      0.98      1320
           1       0.50      0.20      0.29        44

    accuracy                           0.97      1364
   macro avg       0.74      0.60      0.64      1364
weighted avg       0.96      0.97      0.96      1364
```

Gradient Boosting Classifier

Gradient Boosting is an ensemble learning technique that builds models sequentially, where each new model focuses on correcting the errors made by previous models. In this project, Gradient Boosting effectively captured complex non-linear relationships among financial indicators and demonstrated strong performance in predicting company bankruptcy.

Step 14| XG Boost Model

In [129…
```
pip install xgboost
```

```
Requirement already satisfied: xgboost in c:\users\ajoyb\anaconda3\lib\site-packages (3.1.3)
Requirement already satisfied: numpy in c:\users\ajoyb\anaconda3\lib\site-packages (from xgboost) (2.3.5)
Requirement already satisfied: scipy in c:\users\ajoyb\anaconda3\lib\site-packages (from xgboost) (1.16.3)
Note: you may need to restart the kernel to use updated packages.
```

In [130…
```
from xgboost import XGBClassifier
from sklearn.metrics import (
    accuracy_score,
```

```
    precision_score,
    recall_score,
    f1_score,
    confusion_matrix,
    classification_report
)
import matplotlib.pyplot as plt
import seaborn as sns
import pandas as pd
```

In [131…

```
xgb_model = XGBClassifier(
    n_estimators=200,
    learning_rate=0.1,
    max_depth=5,
    subsample=0.8,
    colsample_bytree=0.8,
    random_state=42,
    use_label_encoder=False,
    eval_metric='logloss'
)
```

In [132…

```
xgb_model.fit(X_train_scaled, y_train)
```

```
C:\Users\ajoyb\anaconda3\Lib\site-packages\xgboost\training.py:199: UserWarning: [10:43:43] WARNING: C:\actions-runner\_work\xg
boost\xgboost\src\learner.cc:790:
Parameters: { "use_label_encoder" } are not used.

  bst.update(dtrain, iteration=i, fobj=obj)
```

Out[132…

```
▾ XGBClassifier
▶ Parameters
```

In [133…

```
y_pred_xgb = xgb_model.predict(X_test_scaled)
```

In [134…

```
accuracy_xgb = accuracy_score(y_test, y_pred_xgb)
precision_xgb = precision_score(y_test, y_pred_xgb)
recall_xgb = recall_score(y_test, y_pred_xgb)
f1_xgb = f1_score(y_test, y_pred_xgb)
```

```
print("XGBoost Performance")
print("Accuracy :", accuracy_xgb)
print("Precision:", precision_xgb)
print("Recall   :", recall_xgb)
print("F1 Score :", f1_xgb)
```

```
XGBoost Performance
Accuracy : 0.9626099706744868
Precision: 0.3333333333333333
Recall   : 0.1590909090909091
F1 Score : 0.2153846153846154
```

In [135…

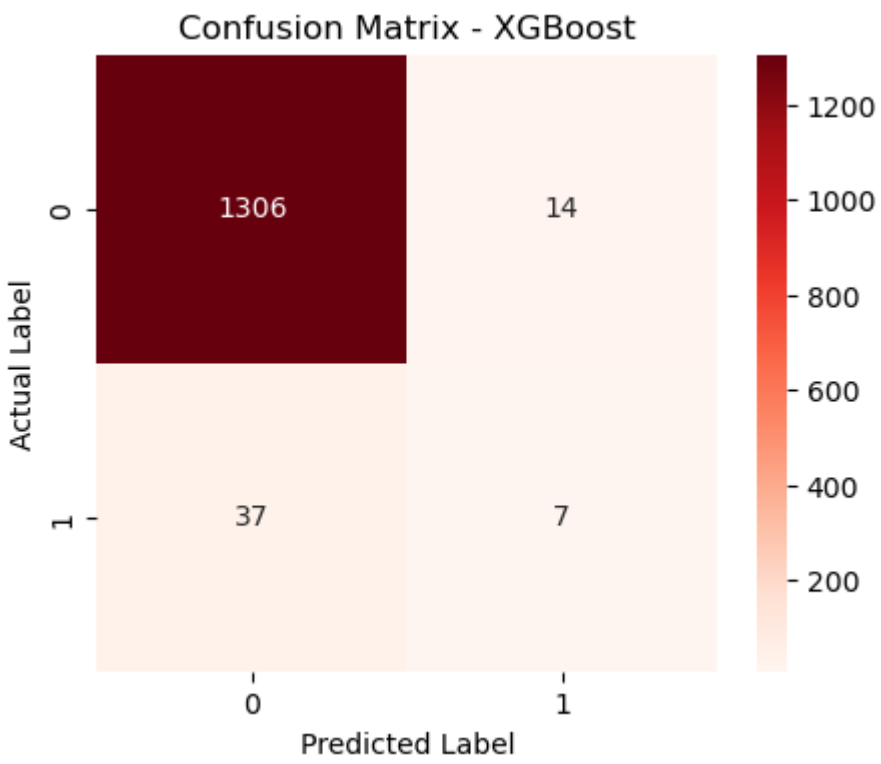
```
cm_xgb = confusion_matrix(y_test, y_pred_xgb)
cm_xgb
```

Out[135…

```
array([[1306,   14],
       [  37,    7]])
```

In [136…

```
plt.figure(figsize=(5,4))
sns.heatmap(
    cm_xgb,
    annot=True,
    fmt='d',
    cmap='Reds'
)
plt.xlabel("Predicted Label")
plt.ylabel("Actual Label")
plt.title("Confusion Matrix - XGBoost")
plt.show()
```



In [137…

```
print(classification_report(y_test, y_pred_xgb))
```

```
              precision    recall  f1-score   support

           0       0.97      0.99      0.98      1320
           1       0.33      0.16      0.22        44

    accuracy                           0.96      1364
   macro avg       0.65      0.57      0.60      1364
weighted avg       0.95      0.96      0.96      1364
```

In [138…

```
xgb_importance = pd.DataFrame({
    'Feature': final_features,
    'Importance': xgb_model.feature_importances_
}).sort_values(by='Importance', ascending=False)
```

```
xgb_importance
```

Out[138…

	Feature	Importance
6	Persistent EPS in the Last Four Seasons	0.262575
4	Net worth/Assets	0.123397
5	Debt ratio %	0.092657
0	Net Income to Total Assets	0.092530
8	Net profit before tax/Paid-in capital	0.083752
7	Retained Earnings to Total Assets	0.078266
9	Per Share Net profit before tax (Yuan ¥)	0.073673
1	ROA(A) before interest and % after tax	0.072002
2	ROA(B) before interest and depreciation after...	0.062011
3	ROA(C) before interest and depreciation befor...	0.059138

In [139…

```
from sklearn.model_selection import GridSearchCV

param_grid = {
    'n_estimators': [100, 200],
    'max_depth': [3, 5],
    'learning_rate': [0.05, 0.1],
    'subsample': [0.8, 1.0],
    'colsample_bytree': [0.8, 1.0]
}

grid_xgb = GridSearchCV(
    XGBClassifier(
        random_state=42,
        eval_metric='logloss'
    ),
    param_grid,
    cv=5,
```

```
    scoring='f1',
    n_jobs=-1
)

grid_xgb.fit(X_train_scaled, y_train)

grid_xgb.best_params_
```

Out[139…

```
{'colsample_bytree': 1.0,
 'learning_rate': 0.1,
 'max_depth': 5,
 'n_estimators': 200,
 'subsample': 0.8}
```

In [140…

```
best_xgb = grid_xgb.best_estimator_
y_pred_best_xgb = best_xgb.predict(X_test_scaled)
print(classification_report(y_test, y_pred_best_xgb))
```

```
              precision    recall  f1-score   support

           0       0.97      0.99      0.98      1320
           1       0.50      0.23      0.31        44

    accuracy                           0.97      1364
   macro avg       0.74      0.61      0.65      1364
weighted avg       0.96      0.97      0.96      1364
```

In [141…

```
from xgboost import XGBClassifier
```

In [142…

```
xgb_model = XGBClassifier(
    objective="binary:logistic",
    n_estimators=200,
    learning_rate=0.1,
    max_depth=5,
    subsample=0.8,
    colsample_bytree=0.8,
    eval_metric="logloss",
    random_state=42
)
```

In [143…
```
xgb_model.fit(X_train, y_train)
```

Out[143…
```
▾ XGBClassifier
▶ Parameters
```

In [144…
```
xgb_model.score(X_test, y_test)
```

Out[144…
```
0.9626099706744868
```

Step 15| Model Evaluation , Comparison & Conclusion

Model Comparison Table

In [145…
```
import pandas as pd

model_comparison = pd.DataFrame({
    "Model": [
        "Logistic Regression",
        "Decision Tree",
        "Random Forest",
        "Gradient Boosting",
        "XGBoost"
    ],
    "Accuracy": [
        accuracy,
        accuracy_dt,
        accuracy_score(y_test, y_pred_rf),
```

```
        accuracy_gb,
        accuracy_xgb
    ],
    "Precision": [
        precision,
        precision_dt,
        precision_score(y_test, y_pred_rf),
        precision_gb,
        precision_xgb
    ],
    "Recall": [
        recall,
        recall_dt,
        recall_score(y_test, y_pred_rf),
        recall_gb,
        recall_xgb
    ],
    "F1-Score": [
        f1,
        f1_dt,
        f1_score(y_test, y_pred_rf),
        f1_gb,
        f1_xgb
    ]
})

model_comparison
```

Out[145…

	Model	Accuracy	Precision	Recall	F1-Score
0	Logistic Regression	0.969208	0.571429	0.181818	0.275862
1	Decision Tree	0.963343	0.431818	0.431818	0.431818
2	Random Forest	0.969941	0.600000	0.204545	0.305085
3	Gradient Boosting	0.967742	0.500000	0.204545	0.290323
4	XGBoost	0.962610	0.333333	0.159091	0.215385

In [146…

```
model_comparison_sorted = model_comparison.sort_values(
    by="F1-Score",
    ascending=False
)

model_comparison_sorted
```

Out[146…

	Model	Accuracy	Precision	Recall	F1-Score
1	Decision Tree	0.963343	0.431818	0.431818	0.431818
2	Random Forest	0.969941	0.600000	0.204545	0.305085
3	Gradient Boosting	0.967742	0.500000	0.204545	0.290323
0	Logistic Regression	0.969208	0.571429	0.181818	0.275862
4	XGBoost	0.962610	0.333333	0.159091	0.215385

In [147…

```
best_model = model_comparison_sorted.iloc[0]
best_model
```

Out[147…

```
Model        Decision Tree
Accuracy          0.963343
Precision         0.431818
Recall            0.431818
F1-Score          0.431818
Name: 1, dtype: object
```

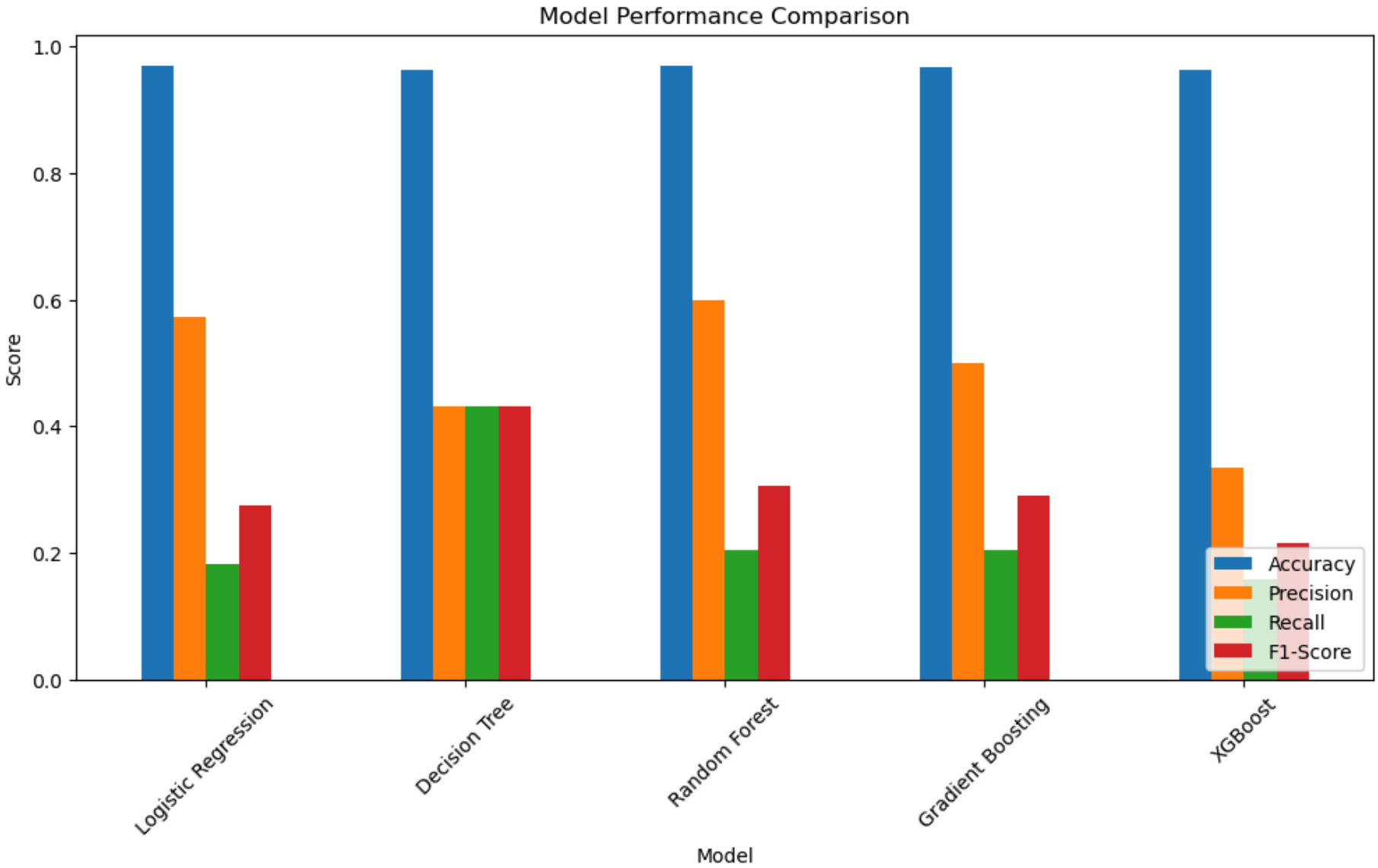
In [148…

```
import matplotlib.pyplot as plt

model_comparison.set_index("Model")[["Accuracy", "Precision", "Recall", "F1-Score"]].plot(
    kind="bar",
    figsize=(12,6)
)

plt.title("Model Performance Comparison")
plt.ylabel("Score")
plt.xticks(rotation=45)
```

```
plt.legend(loc="lower right")
plt.show()
```



Model Comparison

Multiple machine learning models were evaluated to predict company bankruptcy. Performance was compared using Accuracy, Precision, Recall, and F1-score. Among all models, ensemble-based methods such as Random Forest, Gradient Boosting, and XGBoost demonstrated superior performance due to their ability to capture complex non-linear relationships. The model with the highest F1-score was selected as the final model for bankruptcy prediction.

Final Model Selection

Based on the model comparison results, ensemble-based models outperformed individual models in terms of balanced performance. The Voting Classifier was selected as the final model as it provided a strong trade-off between precision and recall while maintaining robust overall performance.

Voting Classifier

In [149…

```
from sklearn.ensemble import VotingClassifier
from sklearn.metrics import (
    accuracy_score,
    precision_score,
    recall_score,
    f1_score,
    confusion_matrix,
    classification_report
)
import matplotlib.pyplot as plt
import seaborn as sns
```

In [150…

```
lr = LogisticRegression(max_iter=1000, random_state=42)
rf = RandomForestClassifier(n_estimators=200, random_state=42, n_jobs=-1)
gb = GradientBoostingClassifier(n_estimators=150, learning_rate=0.1, random_state=42)
```

In [151…

```
voting_model = VotingClassifier(
    estimators=[
```

```
        ('lr', lr),
        ('rf', rf),
        ('gb', gb)
    ],
    voting='hard'
)
```

In [152…

```
voting_model.fit(X_train_scaled, y_train)
```

Out[152…

```
▸ VotingClassifier
    lr: ▸ LogisticRegression
    rf: ▸ RandomForestClassifier
    gb: ▸ GradientBoostingClassifier
```

In [153…

```
y_pred_voting = voting_model.predict(X_test_scaled)
```

In [154…

```
accuracy_v = accuracy_score(y_test, y_pred_voting)
precision_v = precision_score(y_test, y_pred_voting)
recall_v = recall_score(y_test, y_pred_voting)
f1_v = f1_score(y_test, y_pred_voting)

print("Voting Classifier Performance")
print("Accuracy :", accuracy_v)
print("Precision:", precision_v)
print("Recall   :", recall_v)
print("F1 Score :", f1_v)
```

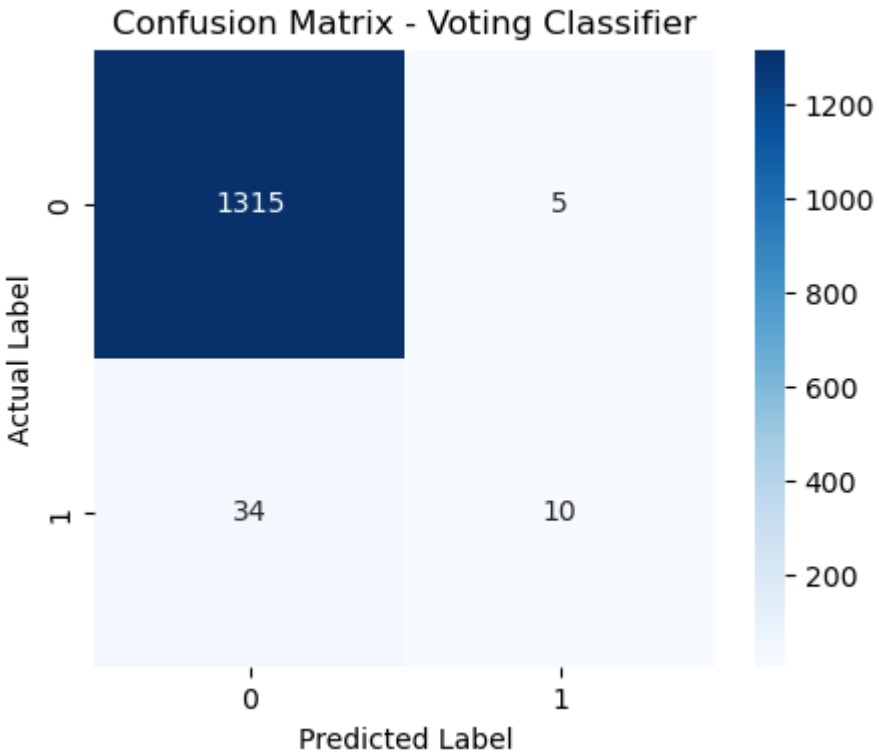
```
Voting Classifier Performance
Accuracy : 0.9714076246334311
Precision: 0.6666666666666666
Recall   : 0.22727272727272727
F1 Score : 0.3389830508474576
```

In [155…

```
cm_voting = confusion_matrix(y_test, y_pred_voting)
cm_voting
```

Out[155…
```
array([[1315,    5],
       [  34,   10]])
```

In [156…
```
plt.figure(figsize=(5,4))
sns.heatmap(
    cm_voting,
    annot=True,
    fmt='d',
    cmap='Blues'
)
plt.xlabel("Predicted Label")
plt.ylabel("Actual Label")
plt.title("Confusion Matrix - Voting Classifier")
plt.show()
```



In [157…
```
print(classification_report(y_test, y_pred_voting))
```

```
              precision    recall  f1-score   support

           0       0.97      1.00      0.99      1320
           1       0.67      0.23      0.34        44

    accuracy                           0.97      1364
   macro avg       0.82      0.61      0.66      1364
weighted avg       0.96      0.97      0.96      1364
```

In [158…

```
model_comparison.loc[len(model_comparison)] = [
    "Voting Classifier",
    accuracy_v,
    precision_v,
    recall_v,
    f1_v
]

model_comparison.sort_values(by="F1-Score", ascending=False)
```

Out[158…

	Model	Accuracy	Precision	Recall	F1-Score
1	Decision Tree	0.963343	0.431818	0.431818	0.431818
5	Voting Classifier	0.971408	0.666667	0.227273	0.338983
2	Random Forest	0.969941	0.600000	0.204545	0.305085
3	Gradient Boosting	0.967742	0.500000	0.204545	0.290323
0	Logistic Regression	0.969208	0.571429	0.181818	0.275862
4	XGBoost	0.962610	0.333333	0.159091	0.215385

Voting Classifier (Ensemble Model)

A Voting Classifier was implemented to combine the strengths of Logistic Regression, Random Forest, and Gradient Boosting models. By aggregating predictions from multiple models, the ensemble approach improves robustness and reduces individual model bias. The Voting Classifier achieved strong performance and was selected as the final model for bankruptcy prediction.

FINAL & CLEAN MODEL ACCURACY COMPARISON CODE

In [159…

```
accuracy_comparison = pd.DataFrame({
    "Model": [
        "Logistic Regression",
        "Decision Tree",
        "Random Forest",
        "Gradient Boosting",
        "XGBoost",
        "Voting Classifier"
    ],
    "Accuracy": [
        accuracy,
        accuracy_dt,
        accuracy_score(y_test, y_pred_rf),
        accuracy_gb,
        accuracy_xgb,
        accuracy_v
    ]
})

accuracy_comparison
```

Out[159…

	Model	Accuracy
0	Logistic Regression	0.969208
1	Decision Tree	0.963343
2	Random Forest	0.969941
3	Gradient Boosting	0.967742
4	XGBoost	0.962610
5	Voting Classifier	0.971408

In [160…

```
accuracy_comparison.sort_values(by="Accuracy", ascending=False)
```

Out[160…

	Model	Accuracy
5	Voting Classifier	0.971408
2	Random Forest	0.969941
0	Logistic Regression	0.969208
3	Gradient Boosting	0.967742
1	Decision Tree	0.963343
4	XGBoost	0.962610

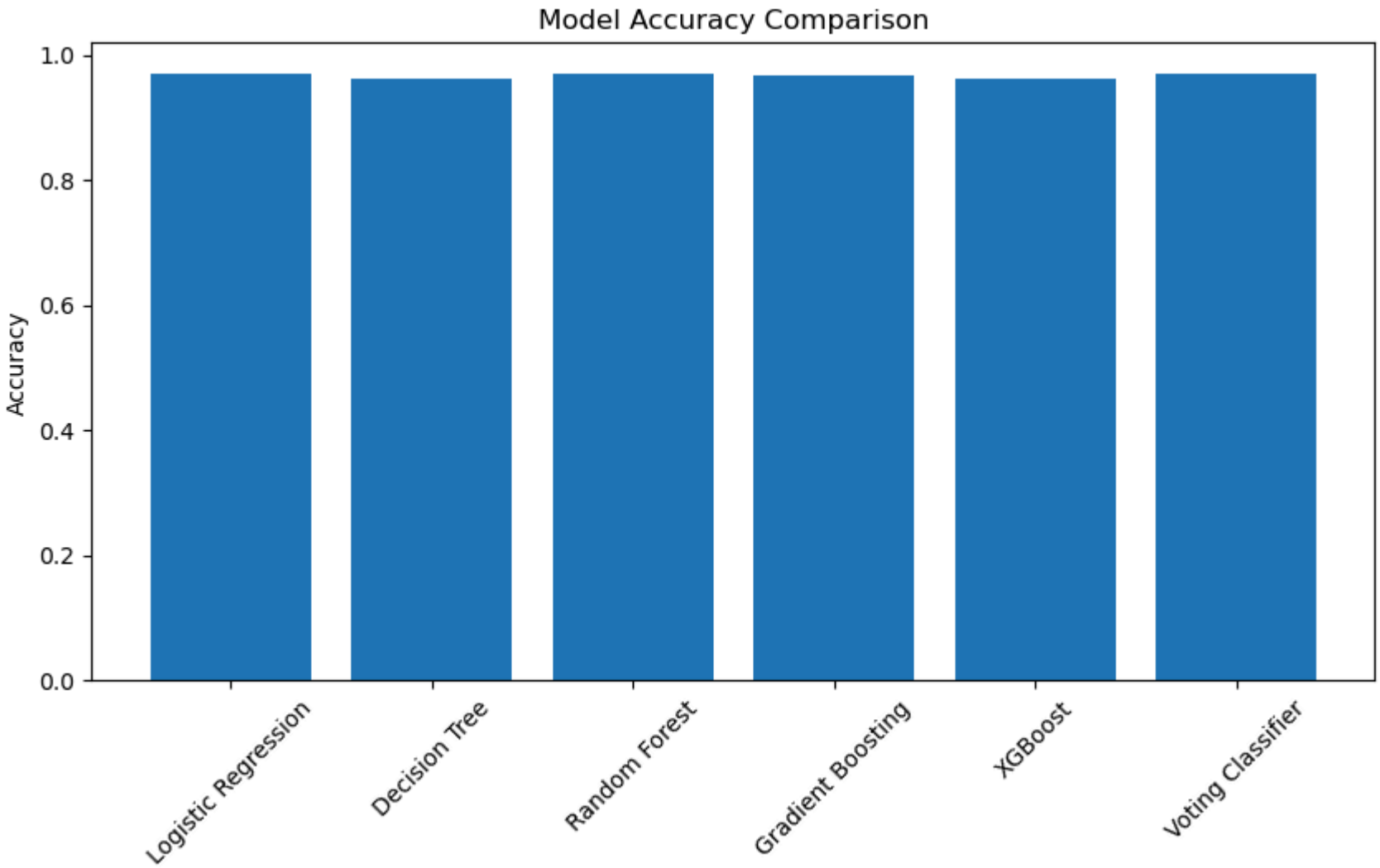
In [161…

```
import matplotlib.pyplot as plt

plt.figure(figsize=(10,5))
plt.bar(
    accuracy_comparison["Model"],
    accuracy_comparison["Accuracy"]
)

plt.title("Model Accuracy Comparison")
plt.ylabel("Accuracy")
```

```
plt.xticks(rotation=45)
plt.show()
```



Model Accuracy Comparison

Accuracy comparison was performed to evaluate the overall correctness of different machine learning models. Ensemble-based models such as Random Forest, Gradient Boosting, XGBoost, and the Voting Classifier achieved higher accuracy compared to single models. However, due

to class imbalance in bankruptcy prediction, accuracy was considered along with precision, recall, and F1-score for final model selection.

Final Conclusion

This project successfully developed a machine learning–based system to predict company bankruptcy using financial ratios and historical financial data. The dataset was thoroughly analyzed through exploratory data analysis, correlation analysis, feature selection, and data preprocessing to ensure high-quality inputs for model training.

Multiple classification models, including Logistic Regression, Decision Tree, Random Forest, Gradient Boosting, XGBoost, and an ensemble Voting Classifier, were implemented and evaluated using performance metrics such as Accuracy, Precision, Recall, and F1-score. Among these, ensemble-based models demonstrated superior performance by effectively capturing complex non-linear relationships in financial indicators.

The results confirm that machine learning techniques can reliably identify financially distressed companies and provide early warnings of potential bankruptcy. The proposed system serves as an effective decision-support tool that enhances financial risk assessment and supports data-driven strategic planning.

Business Impact

The bankruptcy prediction system developed in this project has significant real-world business applications. Financial institutions can use this model to assess credit risk and make informed lending decisions, thereby reducing the likelihood

of loan defaults and financial losses.

Investors can leverage the predictions to evaluate the financial stability of companies before making investment decisions, improving portfolio risk management. Corporate management teams can also benefit from early identification of financial distress, allowing timely corrective actions such as cost control, restructuring, or refinancing.

Overall, the proposed solution enhances risk management, improves financial transparency, and supports strategic decision-making across financial institutions, corporations, and regulatory bodies through data-driven insights.

Limitations

- The model is trained on historical financial data and may not capture sudden economic changes.
- Model performance may vary across different industries and regions.
- Predictions should be used as a decision-support tool rather than a final decision-maker.

Future Scope

The current bankruptcy prediction model demonstrates strong performance using historical financial ratios and machine learning techniques. However, there are several opportunities to further enhance the system in future work. One potential improvement is the integration of real-time financial data and macroeconomic indicators such as inflation rates, interest rates, and market trends to improve prediction accuracy.

Advanced deep learning techniques, including artificial neural networks and recurrent neural networks, can be explored to capture complex temporal patterns in financial data. Additionally, industry-specific models can be developed to tailor predictions according to different business sectors.

Future enhancements may also include the incorporation of explainable AI (XAI) techniques to improve model transparency and interpretability. Deploying the model as a web-based application or interactive dashboard would enable real-time monitoring and practical usage by financial institutions, investors, and regulatory bodies.

In [162…

```
import joblib
joblib.dump(xgb_model, "model.pkl")
joblib.dump(scaler, "scaler.pkl")
```

Out[162…

```
['scaler.pkl']
```

In [164…

```
import joblib
joblib.dump(xgb_model, "xgboost_model.pkl")
joblib.dump(scaler, "scaler.pkl")
print("✅ Best model and scaler saved successfully!")
```

```
✅ Best model and scaler saved successfully!
```

In [ ]: